technische universität
dortmund

Master Thesis

Investigating Ensemble Strategies for Enhancing the Generalization of Foundation Models in Outlier Detection

Phuong Huong Nguyen
05.2026

Lecturers:
Prof. Dr. Emmanuel Müller
Dr. Simon Klüttermann

Technical University of Dortmund
Department of Computer Science
Chair of Data Science and Data Engineering
https://ls9-www.cs.tu-dortmund.de/

Contents

Abstract **2**

Acknowledgements **3**

1 Introduction **4**

2 Related Work **6**

2.1 Outlier Detection in Tabular Data 6

2.2 Transformer 8

2.3 Foundation Models 10

2.3.1 Prior-Data Fitted Networks (PFNs) 11

2.3.2 TabPFN 12

2.3.3 FoMo-0D 13

2.4 Parameter-Efficient Fine-Tuning for Foundation Models 15

2.5 Ensemble Learning 16

2.6 Evaluation Methods 18

2.6.1 Area under ROC curve 18

2.6.2 Area under Precision-Recall curve 19

2.6.3 F1 score 20

3 Methodology **22**

3.1 A Data Prior for Outlier Detection 22

3.1.1 Gaussian Mixture Model 22

3.1.2 Modified Gaussian Mixture Model 23

3.1.3 Cube Mixture Model 24

3.2 Strategies for Optimization of Foundation Models 25

3.3 Ensemble Strategies 26

3.3.1 Ensemble Strategy One - Bagging 27

3.3.2 Ensemble Strategy Two - Selected Bagging 28

3.3.3 Ensemble Strategy Three - Weight Average 29

4 Experiments and Evaluation **31**
4.1 Experiment Setup . . . 31
4.1.1 Benchmark Datasets . . . 31
4.1.2 Baseline Methods . . . 33
4.1.3 Choice of Hyperparameters for Optimization . . . 33
4.2 Experiment Results . . . 34
4.2.1 Performance of Base Models . . . 34
4.2.1.1 Impact of Data Priors on Performance . . . 34
4.2.1.2 Should we optimize max_feature_dim and max_num_cluster? 35
4.2.1.3 Optimization of Learning Rate . . . 36
4.2.1.4 How many Epochs are enough? . . . 37
4.2.1.5 Which batch size is reasonable? . . . 38
4.2.1.6 Impact of Layers and Attention Heads . . . 39
4.2.1.7 Does our base model performs better than the original FoMo-0D? . . . 41
4.2.2 Performance of Ensemble Strategies . . . 43
4.2.2.1 Which combination function is best? . . . 43
4.2.2.2 Which Ensemble Strategies outperform? . . . 43
4.2.2.3 Do large ensembles perform better? . . . 45
4.2.2.4 How does the diversity of base models affect on ensemble performance? . . . 45
4.2.2.5 Comparison between our top ensembles and baseline methods 46
4.2.2.6 The two-sided Wilcoxon signed rank test . . . 47

5 Conclusion and future work **49**

List of Figures **53**

List of Tables **54**

Bibliography **62**

Appendix **62**

A **64**
A.1 p-values . . . 64
A.2 Average AUROC, AUCPR, and F1 Scores . . . 64
A.2.1 Average AUROC Scores . . . 65
A.2.2 Average AUCPR scores . . . 67
A.2.3 Average F1 scores . . . 69

Abstract

Tabular outlier detection (OD) has become increasingly important due to its broad applicability across numerous domains. As datasets continue to grow in scale and complexity, there is an increasing demand for robust and efficient detection methods capable of adapting to evolving and previously unseen anomalies while performing inference in real time. The introduction of FoMo-0D [63], a prior-data fitted Foundation Model for zero-shot tabular outlier detection, opens a paradigm shift for outlier detection in practice. Motivated by the need for scalable and efficient solutions, this work presents the first study investigating ensemble strategies based on the FoMo-0D architecture for tabular outlier detection. We propose a two-stage framework that first optimizes FoMo-0D-based foundation models and subsequently integrates them into ensembles. We explore three ensemble strategies: **Bagging**, **Selected Bagging**, and **Weight Average**. The experiments indicate that the optimized base models can outperform the original FoMo-0D while achieving significant improvements in inference time at a small computational cost. Moreover, the proposed Bagging and Selected Bagging ensembles perform competitively with, or even surpass, all 27 baselines across 57 ADBench datasets while reducing substantial inference latency. These results showcase the scalability, robustness, and strong generalization capability of foundation model ensembles for tabular outlier detection in practice.

Acknowledgements

I would like to express my sincere gratitude to my academic supervisors: **Prof. Dr. Emmanuel Müller** and **Dr. Simon Klüttermann**, for the valuable guidance, continuous support, and constructive feedback throughout the development of this master's thesis.
Finally, my deepest appreciation goes to my family and friends for their countless support and patience throughout my studies.
It should be noted that the work utilizes ChatGPT by OpenAI solely for enhancing the grammar of the text.

Chapter 1

Introduction

Tabular data has become one of the most common data type in real-world machine learning applications [15]. This has led to an increasing demand for the outlier detection (OD) in tabular data, which refers to the task of identifying records in a dataset that are most unusual, either relative to other records in the same dataset or relative to previously observed data. Tabular outlier detection plays a crucial role in safeguarding data integrity in critical areas such as cybersecurity [12], financial fraud [13], environmental monitoring [24], and healthcare [69], where outliers can lead to serious operational and economic consequences.

The growing scale and complexity of datasets have challenged the traditional machine learning techniques for outlier detection. Although deep anomaly detection methods have demonstrated capabilities in addressing complex real-world detection tasks [55], their performance is often highly sensitive to hyperparameter configurations [18]. This sensitivity complicates model selection and hinders fair comparison among various models.

As foundation models have achieved superior performance in learning from textual and visual data, many researchers have begun to apply them to tabular data. Notably, FoMo-0D [63] has been introduced recently, marking it as the first pre-trained foundation model for tabular OD. FoMo-0D is a prior-data fitted Foundation Model for zero-shot Outlier Detection on unseen tabular data. After being pre-trained on large-scale synthetic data, it can directly make predictions on new datasets without requiring hyperparameter fine-tuning or additional training. The method addresses challenges related to the dynamic, continuous evolution of data and model selection, making it potentially suitable for real-world applications.

Towards the broader objective of developing deployable machine learning solutions, it is necessary to have robust and efficient outlier detection methods that can adapt to evolving and previously unseen anomalies, and perform inference in real time. Building upon the success of FoMo-0D, our work represents the first attempt to investigate ensemble strategies to further boost the generalization capability of foundation models in tabular

outlier detection, aiming to develop robust and efficient solutions suitable for deployment in real-world applications.

Unlike conventional machine learning models, foundation models typically contain a large number of parameters and require extensive computational resources during both training and inference. Consequently, constructing ensembles of foundation models introduces additional challenges related to computational overhead and inference latency. To address these issues, we propose a two-stage framework for ensemble construction. In the first stage, we optimize FoMo-0D-based foundation models to improve computational efficiency while achieving comparable or even enhanced performance relative to the original FoMo-0D model. In the second stage, these models are combined through a unified decision-making mechanism. Here, we investigate three different ensemble strategies, which we call as **Bagging**, **Selected Bagging**, and **Weight Average**.

The following chapters are organized as follows:

- Section 2 introduces the theoretical background of this work, including outlier detection in tabular data, the transformer architecture on which FoMo-0D is built, foundation models and their fine-tuning techniques, ensemble learning principles, and commonly used evaluation metrics for unsupervised outlier detection.

- Section 3 discusses the methodology for constructing ensembles of foundation models for outlier detection. In this section, we first explore the data priors employed to generate diverse synthetic datasets for pre-training foundation models. Then, we explain the two stages of the ensemble construction framework in detail.

- Section 4 details the experimental work, including benchmark datasets, baseline methods, the choice of hyperparameters for optimization, along with the experimental results.

- Section 5 briefs the conclusions and outlines potential directions for future research.

Chapter 2

Related Work

This chapter reviews existing research on outlier detection methods and foundation models in the context of unsupervised outlier detection for tabular data. In addition, we discuss ensemble learning techniques and commonly used evaluation methodologies for anomaly detection models in unsupervised setting.

2.1 Outlier Detection in Tabular Data

Outlier detection has been studied for centuries and continues to be a fundamental topic in the research community. Many definitions of an outlier have been proposed in the literature. One of the most widely cited definitions, introduced by Barnett and Lewis (1994), describes an outlier as an observation (or a subset of observations) that appears to be inconsistent with the remainder of the dataset [6]. From an intuitive perspective, an outlier can be understood as any data instance that exhibits a substantial deviation from the dominant patterns present in the data.

Any data with high variability of data types and formats is categorized as heterogeneous data [71]. Such data are often generated from the Internet of Things (IoT), where network connectivity and computational capabilities extend beyond traditional computing devices to include sensors, physical objects, and everyday items [60]. Due to missing values, high data redundancy, and untruthfulness, heterogeneous data are often ambiguous and low quality [71]. However, they represent the most prevalent form of real-world data and are essential for a wide range of practical applications. Tabular data are a subcategory of heterogeneous data. They are typically presented in a tabular format, where data instances are represented as rows and features as columns. Figure 2.1 shows an example of a heterogeneous tabular dataset containing both numerical and categorical features.

Outlier detection in tabular data refers to the task of identifying records in a dataset that are most unusual, either relative to other records in the same dataset or relative to previously observed data. Although deep learning methods have achieved superior performance

Age	Education	Occupation	Sex	Income
39	Bachelors	Adm-clerical	Male	≤50K
50	Bachelors	Exec-managerial	Male	>50K
38	HS-grad	Handlers-cleaners	Male	≤50K
53	11th	Handlers-cleaners	Male	≤50K
28	Bachelors	Prof-specialty	Female	>50K

Figure 2.1: An example of a heterogeneous tabular data, whose variables are selected from the adult dataset [19], [9]

in classification and data generation tasks on homogeneous data such as images, audio, and text [9], they still struggle to effectively address outlier detection in tabular data. There are several possible reasons why learning from tabular data remains challenging for deep learning approaches.

- **Low-Quality Data**: Real-world tabular datasets usually contain a variety of feature types, such as numerical, ordinal, and categorical features, as well as missing values and outliers. These characteristics often lead to sparse feature representations, where many values are zero or missing. Moreover, real-world tabular data can have hundreds of dimensions. In high-dimensional spaces, data become increasingly sparse, and this sparsity implies that nearly every data point can be considered an equally plausible outlier from the perspective of proximity-based definitions [2].

- **Missing or Complex Feature Correlations**: The spatial correlations between features in tabular datasets are often missing, or the dependencies between variables are complex and irregular [9]. As a result, deep learning models must learn the structure and relationships between features from scratch when dealing with tabular data.

- **Influence of Preprocessing Strategies**: The performance of deep learning models on tabular data is highly sensitive to preprocessing choices. Encoding categorical variables can produce extremely sparse feature representations or impose artificial ordinal relationships on inherently unordered categories [28]. Furthermore, certain preprocessing steps may discard informative patterns, leading to information loss and diminished predictive performance [21].

The introduction of the transformer architecture [70] has revolutionized many areas of deep learning, significantly contributing to a new scientific and technological revolution in modern life, the era of artificial intelligence. As transformer-based approaches have achieved superior performance in learning from textual and visual data, researchers have begun to apply transformer architectures to tabular data. Following the introduction of the deep tabular data learning architecture, TabNet [5], many studies have focused on

applying transformer-based models to address problems involving tabular data, including outlier detection in tabular datasets.

2.2 Transformer

Since being proposed in 2017, the transformer architecture [70] has become the dominant architecture for language-based foundation models, building upon and extending the success of earlier sequence-to-sequence architectures [67]. A Sequence-to-sequence architecture consists of an encoder that processes input data and a decoder that is responsible for generating outputs. Both the inputs and outputs are represented as sequences of tokens. At its core, the encoder processes the input tokens sequentially and produces a final hidden state that represents the encoded input for the decoder [42]. The decoder then takes the final hidden state along with the previously generated token as inputs to generate output tokens sequentially. The transformer addresses two major limitations inherent in the sequence-to-sequence architecture. First, the quality of generated outputs is constrained because the decoder in the sequence-to-sequence architecture relies solely on the final hidden state of the initial input to generate output tokens [42]. Intuitively, this is analogous to generating answers about a book using only its summary rather than having access to the entire content. Second, as both input processing in the encoder and output generation in the decoder are performed sequentially, the overall process becomes slow for long input sequences containing many tokens.

Figure 2.2 demonstrates the architecture of the transformer, which follows an encoder–decoder architecture. The encoder converts an input sequence $x_1, \ldots, x_n$ into a sequence of continuous representations $z_1, \ldots, z_n$, which is then transferred to the decoder to produce an output sequence $y_1, \ldots, y_m$ [70]. At each decoding step, the model operates in an autoregressive manner [30], generating the output sequence sequentially by predicting the next token based on all the tokens that came before it. The following provides a brief overview of its most important components.

Encoder

The encoder is a stack consisting of six identical layers with two sub-layers for each layer. While the first sub-layer is a multi-head self-attention mechanism, the second is a feed-forward network [70]. A residual connection [36] is employed around each of the two sub-layers, followed by layer normalization [70]. The residual connection is a technique in neural networks that add the input of a layer directly to its output, creating a shortcut that bypasses one or more layers.

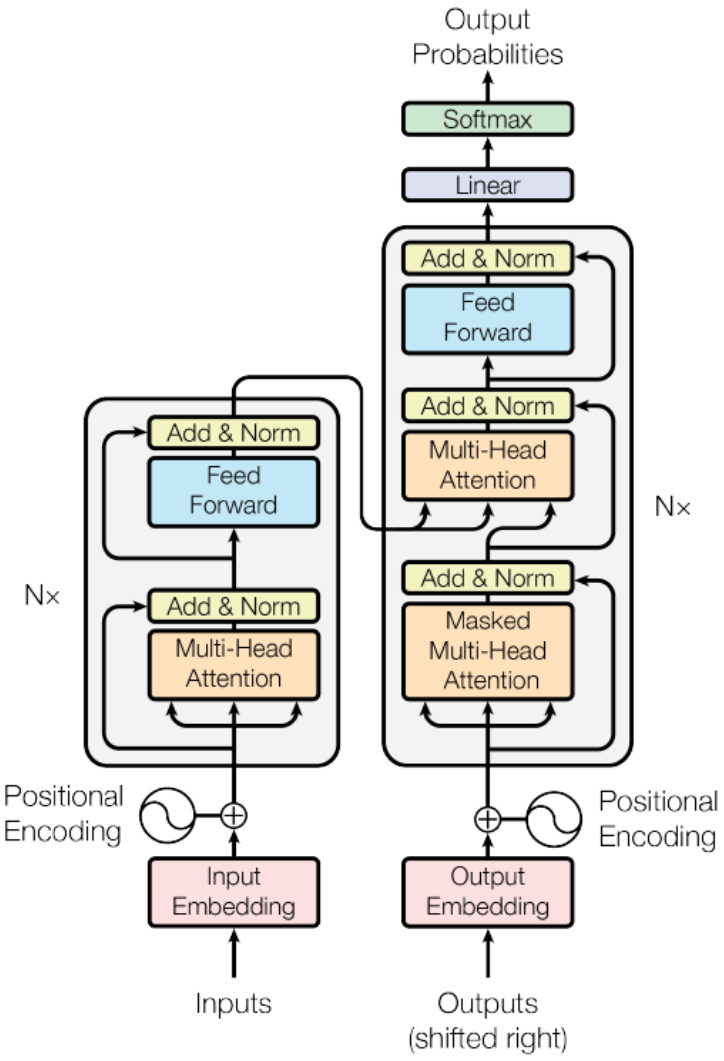


Figure 2.2: Transformer model architecture [70]

Decoder

Similar to the encoder, the decoder is also a stack of six identical layers. In each layer, in addition to the two sub-layers used in the encoder, a third sub-layer is included that performs multi-head attention. Furthermore, the self-attention mechanism within the decoder is modified through masking to ensure that predictions at position i depend only on previous generated outputs at positions less than i, thereby preserving the auto-regressive property of the model [70]. Masking in the transformer is a technique used to control which tokens a model is allowed to see when computing attention.

Attention Mechanism

At the core of the transformer architecture is the attention mechanism, which determines how the model focuses its attention on a piece of text to identify which parts are more or less important. The attention mechanism maps a query vector and a set of key-value vector pairs to an output [70]. In particular, the query vector (Q) represents the current state of the decoder at a given decoding step [42]. Each key vector (K) corresponds to a previous token, including both input tokens and previously generated tokens, while each value vector (V) represents the corresponding content of that token as learned by the model [42]. To compute how much attention to give an input token, the mechanism performs a dot product between the query vector and its key vector [42]. A higher score indicates that the model will use more of the associated value vector when generating the output token.

Feed-Forward Networks

Each layer in both the encoder and decoder contains a fully connected feed-forward network, which is applied independently and identically to each position in the input or output sequence [70]. Each feed-forward network contains two linear transformations with a ReLU activation function in between, enabling the model to learn nonlinear patterns.

Embeddings and Softmax

An embedding module is applied before the transformer blocks to convert the input tokens and output tokens to vectors. Moreover, a linear transformation and a softmax function are used at the end of the decoder block to map output vectors into token probabilities used to sample model outputs [70].

Positional Encoding

To ensure that the model can utilize the order of a sequence, information about the relative or absolute positions of tokens in the sequence must be injected [70]. Therefore, positional encodings are added immediately after the embedding layer at the bottom of both the encoder and decoder stacks.

2.3 Foundation Models

A foundation model is a machine learning model that is trained on a wide range of data and can subsequently be adapted (e.g., fine-tuned) to a large variety of downstream tasks related to the original training model [8]. Foundation models have existed for decades, as they are built upon deep neural networks and self-supervised learning paradigms. However, the beginning of the modern foundation model era is marked by breakthroughs in natural

language processing (NLP) around the end of 2018. Foundation models are enabled by transfer learning [68] and scale [8]. While transfer learning enables foundation models to utilize knowledge acquired from one task for other tasks, scale is a key factor that makes them powerful. This scaling effect is driven by three main elements: advances in computing hardware, the development of the transformer architecture that exploits parallel computation to train more expressive models, and the availability of significantly larger training datasets [8].

As foundation models have achieved superior performance in learning from textual and visual data, many researchers have begun to apply them to tabular data. One of the most prominent approaches is Prior-Data Fitted Networks (PFNs) [54], which marked a milestone in learning on tabular data by introducing the core idea of computing a posterior predictive distribution (PPD) for a test point given the training data as context. This approach has inspired the development of models such as TabPFN [39], and FoMo-0D [63], which address learning problems on tabular data.

2.3.1 Prior-Data Fitted Networks (PFNs)

Prior-Data Fitted Networks (PFNs) [54] are built on a transformer encoder architecture without positional encodings and incorporate a novel regression head for regression tasks. The model is trained to approximate the posterior predictive distribution (PPD) for arbitrary priors from which data are sampled. PFNs have achieved significant results across various domains, including Gaussian process regression, Bayesian neural networks, classification on small tabular datasets, and few-shot image classification [54]. Figure 2.3 illustrates the PFN framework for approximating such Bayesian models.

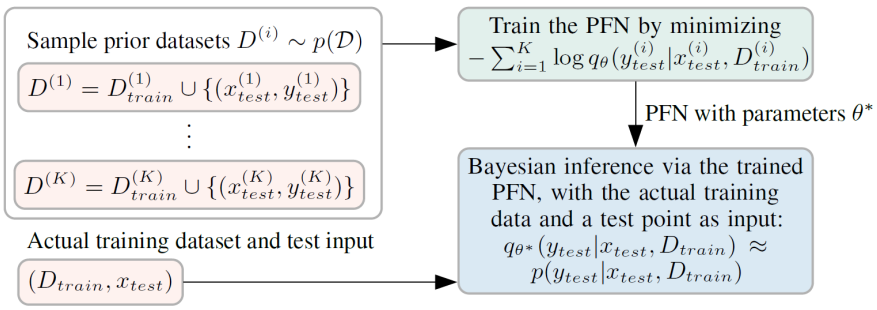


Figure 2.3: The outline of Prior-Data Fitted Networks [54]

A PFN consists of two stages: pre-training on a large amount of synthetic data and inference on real-world data. During training, a set of data points and their corresponding labels is repeatedly sampled from a given prior distribution. One of the labels is then masked, and the model learns to make probabilistic predictions for the masked label based on the set-valued input of the remaining data points [54]. During inference, the pre-trained PFN performs in-context learning (ICL), meaning that it learns to make predictions from

sequences of labeled examples presented in the input without further parameter updates [4]. Specifically, the model is fed with a test point from a real-world dataset along with a set of training samples as inputs, and it outputs a prediction distribution for the test point, without requiring parameter fine-tuning or model selection on new datasets.
In the Bayesian framework for supervised learning, the prior defines a space of hypotheses Φ, which expresses our beliefs about the data distribution before observing any data [63]. Each hypothesis $\phi \in \Phi$ describes a data distribution from which a dataset can be sampled. For instance, given a prior based on Gaussian Mixture Models (GMMs), Φ represents the space of all GMMs, a hypothesis ϕ corresponds to a specific GMM, and a dataset is obtained by sampling from this GMM. The posterior predictive distribution $p(y_{test}|x_{test}, D_{train})$ for a new, unseen test point x_{test} specifies the distribution of its label conditioned on the training data $D_{train} = (x_1, y_1), ..., (x_n, y_n)$. The PPD can be obtained by integrating over the hypothesis space Φ, where $p(\phi)$ denotes the prior probability and $p(D_{train}|\phi)$ denotes the likelihood of the training data given ϕ [63]:

$$p(y_{test}|x_{test}, D_{train}) = \int_{\Phi} p(y_{test}|x_{test}, \phi)p(D_{train}|\phi)p(\phi)d_{\phi} \tag{2.1}$$

2.3.2 TabPFN

TabPFN [39] is a foundation model designed to address small tabular classification tasks in less than a second without requiring hyperparameter fine-tuning. TabPFN is built on Prior-Data Fitted Networks with two key modifications: the attention masks to reduce inference time, and a flexible encoder that enables the model to operate on datasets with varying numbers of features [39]. In particular, whereas the original PFN architecture employs a single multi-head self-attention module, TabPFN uses two attention modules with shared weights. One module computes self-attention among the training examples, while the other performs cross-attention from validation examples to training examples [39]. Additionally, while the encoder in a PFN architecture accepts inputs with a fixed dimensionality, TabPFN adopts a flexible encoder that accommodates datasets with different numbers of features through zero-padding. Specifically, when the number of features k is smaller than the maximum number of features K, the inputs are scaled by a factor of $\frac{K}{k}$ to preserve the magnitude of the representations.
To generate diverse data in priors, TabPFN uses Structural Causal Models (SCMs) and Bayesian Neural Networks (BNNs) as fundamental mechanisms [39]. SCMs model causal relationships between variables through causal graphs and structural assignments. An SCM is specified by a directed acyclic graph $G = (V, E)$, where each node $i \in V$ represents a variable and directed edges indicate causal relationships from causes to effects. Each mechanism z_i in a collection $Z := (\{z_1, ..., z_k\})$ of structural assignments is associated with a node in G. Each z_i is defined as $z_i = f_i(z_{PA_G(i)}, \epsilon_i)$, where $PA_G(i)$ denotes the set of parent variables of node i (i.e., its direct causes) in the causal graph G, f_i is a potentially

non-linear deterministic function and ϵ_i represents a noise variable [39]. A BNN is defined as a stochastic artificial neural network trained using a Bayesian inference [44]. To generate a dataset from a BNN prior, TabPFN first samples a neural network architecture and along with its weights. Then, each input x is sampled and fed through the BNN together with sampled noise variables [39], producing an output y that is used as the target. During PFN training, datasets generated from SCM and BNN priors are sampled randomly from either mechanism with equal probability.

2.3.3 FoMo-0D

FoMo-0D [63] is a prior-data fitted Foundation Model for zero-shot Outlier Detection on unseen tabular data. Zero-shot Outlier Detection task refers to identifying anomaly regions without using target domain data [8]. After being pre-trained on large-scale synthetic data, FoMo-0D can directly make predictions on new samples without requiring hyperparameter fine-tuning or additional training. Figure 2.4 demonstrates the difference between a typical outlier detection (OD) model and the FoMo-0D model. A conventional OD model is trained directly on an observed dataset and can only perform inference on the remaining portion of the same dataset that was not used for training. Meanwhile, the FoMo-0D is pre-trained once on simulated datasets generated according to a prior distribution and can subsequently perform inference on arbitrary real-world datasets.

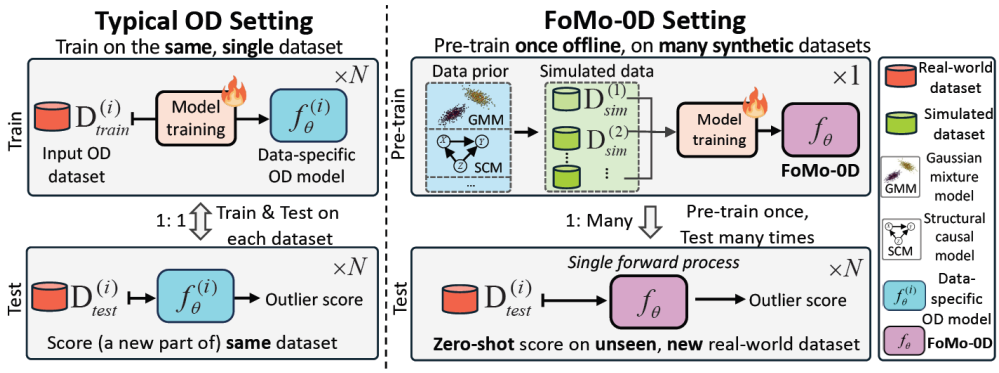


Figure 2.4: Comparison of typical OD vs. the FoMo-0D settings [63]

During the pre-training stage of FoMo-0D, large-scale synthetic datasets are generated by first sampling a hypothesis $\phi \sim p(\phi)$, and then sampling a dataset $D \sim p(D|\phi)$. In particular, a prior based on Gaussian Mixture Models (GMMs) is employed. Each dataset D is drawn randomly from a various GMM configuration. Subsequently, each dataset D is partitioned into a test subset $D_{test} \subset D$ and a training subset $D_{train} = D \setminus D_{test}$. Data points $(x_{test}, y_{test}) \in D_{test}$ are used to optimize FoMo-0D's parameters θ based on the following training loss [63]:

$$L = E_{(\{(x_{test},y_{test})\}\cup D_{train})\sim p(D)}[-\log q_\theta(y_{test}|x_{test}, D_{train})] \tag{2.2}$$

At the inference stage, the pre-trained FoMo-0D can be applied to any real-world dataset [63] without hyperparameter fine-tuning or additional training. By feeding $< D_{train}, x_{test} >$ as input into the FoMo-0D model, where D_{train} is a real-world training dataset and $x_{test} \in D_{test}$, the PPD $q_\theta(y|x_{test}, D_{train})$ is computed in a single forward pass. This approach leverages the in-context learning technique, which enables the model to generate a predictive distribution for a test point based on the test instance and a set of training samples.

Similar to TabPFN, FoMo-0D is built on Prior-Data Fitted Networks, encoding the feature vector of each sample as a token and allowing token presentations to attend to each other [63]. FoMo-0D also adopts the architectural customizations introduced in TabPFN. Specifically, the model employs two attention modules: one computes self-attention among the training samples, and the other performs cross-attention from test samples to training samples. Moreover, a flexible encoder is used to handle varying feature dimensionalities via zero-padding. To obtain model invariance in the dataset, input samples are randomly permuted.

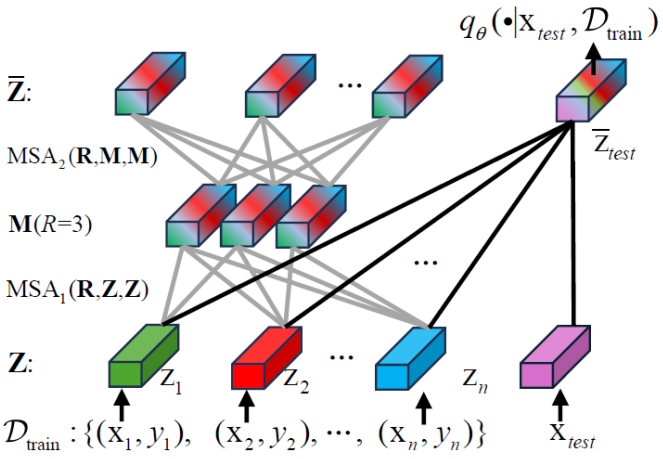


Figure 2.5: The router mechanism for scalable attention employed in FoMo-0D architecture [63]

To reduce the computational complexity during the pre-training stage, FoMo-0D's attention is scaled up via the router mechanism introduced in [80]. Figure 2.5 illustrates the router mechanism employed in the FoMo-0D architecture. The core idea is to learn a small number of routers or representatives $R \in \mathbb{R}^{R \times d}$, which aggregate information from all n samples and then distribute it back to the n output embeddings [63]. Particularly, the representatives R first gather information from all n samples by serving as the query in the multi-head self-attention (MSA) [63]:

$$\mathcal{M} = MSA_1(R, Z, Z) \tag{2.3}$$

where $\mathcal{M}$ denotes the aggregated messages and Z represents output embeddings of n samples from self-attention layers. Subsequently, the representatives distribute the aggregated

information by applying the sample embeddings Z as query and the aggregated messages $\mathcal{M}$ as both key and value [63]:

$$\hat{Z} = MSA_2(Z, \mathcal{M}, \mathcal{M}) \tag{2.4}$$

The final embeddings of the n training samples are obtained as $\overline{Z} = LayerNorm(\hat{Z} + Z)$. This mechanism reduces the computational complexity of attention during pre-training from $\mathcal{O}(n^2)$ to $\mathcal{O}(2Rn) = \mathcal{O}(n)$ [63].

2.4 Parameter-Efficient Fine-Tuning for Foundation Models

Fine-tuning of foundation models aims to efficiently adapt large pre-trained models to specific downstream tasks. However, foundation models often contain billions of parameters and require substantial computational resources for training and inference, which creates challenges when adapting them to specific tasks. To address this issue, Parameter-Efficient Fine-Tuning (PEFT) methods [33, 76, 81] have been proposed. This technique involves selectively fine-tuning a small subset of a foundation model's parameters while keeping the remaining parameters unchanged. This approach aims to reduce the number of trainable parameters and computational complexity required when customizing foundation models for specific downstream tasks [79]. A variety of PEFT techniques have been introduced to accommodate the diversity of foundation model architectures. Below, we briefly summarize several important categories of PEFT methods.

- **Selective PEFT**: Approaches in this category refer to either fine-tuning only a subset of the existing parameters while keeping the remaining frozen, or introducing a small number of additional trainable parameters [79]. The underlying assumption of this technique is that certain parameters in the pre-training models are particularly crucial for a specific task. Therefore, adjusting these parameters can produce satisfactory performance while reducing computational requirements.

- **Additive PEFT**: Additive methods involve inserting adapter modules between layers of a foundation model. These adapter layers are designed to be smaller than those of the original network and are initialized close to a near-identity function to ensure stable training of the adapted model [41]. During fine-tuning for a specific downstream task, only the parameters of the adapter layers are updated, while the parameters of the pre-trained model remain frozen. As a result, the requirements of storage, memory, and computational resource are significantly reduced.

- **Prompt PEFT**: This category represents one of the most common PEFT methods, particularly in the context of large language models. The approach involves incorporating specially designed prompts into the input or into the layers of a transformer

model. The goal is to align the input distribution with the distribution of the original training data and guide the model toward producing the desired output [79].

- **Reparameterization PEFT**: Reparameterization refers to an equivalent transformation of a model's architecture from one to another by transforming its parameters [33]. In PEFT, reparameterization-based methods optimize a low-dimensional representation of the original model parameters during training. The learned parameters are then merged back into the original weight parameterization during inference.

- **Hybrid PEFT**: Methods in this category combine several PEFT strategies to leverage their complementary strengths. Such approaches either integrate the advantages of different PEFT techniques or aim to develop a unified framework by analyzing the similarities among these methods [33].

2.5 Ensemble Learning

Ensemble methods are widely used in many data mining and machine learning tasks, including outlier detection, to reduce the dependence of the final prediction on a specific data locality or a specific model choice. Rather than relying on a single model, ensemble learning combines multiple algorithms (also known as base models) to produce a final decision, often resulting in more accurate and robust predictions. The core idea is that by integrating diverse base models, the errors made by individual models can be compensated for by others, allowing the overall prediction performance of the ensemble to surpass that of any base model.

To construct a good ensemble, each base model should be accurate and diverse [34]. The accuracy means each base model has an error rate better than random guessing on new data, while the diversity ensures all models will not make the same errors on the test examples [17]. Training an ensemble contains two steps: creating a set of base models from the training data, and then combining them to get the final composite predictor [35].

In terms of creating base models, ensemble methods can employ either data-centric or model-centric approaches [1]. In data-centric ensembles, base models are derived from the same algorithm but are trained on different variants of a given dataset. These dataset variants can be generated through various techniques, such as transforming the original dataset, drawing samples to create new datasets, or introducing noise into the data. In contrast, model-centric ensembles construct base models using different algorithms, different parameter configurations of the same algorithm, or diverse randomized initializations of the same model.

In terms of combining base models, they can be executed independently of each other, with all predictions subsequently combined into the final output. Bagging [10] and stacking [74] algorithms are a few of ensemble methods whose components are independent of each

other. Alternatively, they can be implemented sequentially, where each base model refines the predictions of its predecessors. With this approach, the final prediction can either be the result of the last base model or a combination of all base learners. Boosting [22] can be categorized as sequential ensemble learning because the execution of a particular base model depends on the outcome of previous ones.

Bias and Variance

In the context of statistical learning theory, a model is referred to as an estimator $\hat{\theta}$ of the true underlying process θ that generates training data [27]. The error of an estimator can be decomposed into two components: bias and variance, which are closely related to overfitting, underfitting, and model capacity concepts in machine learning. The bias, defined as $bias(\hat{\theta}) = \mathbb{E}(\hat{\theta}) - \theta$, represents the expected deviation of the estimator from the true value. The variance, $Var(\hat{\theta})$, measures the variability of the estimator across different training datasets sampled from the same underlying data-generating process. A model with high bias indicates that it does not adequately fit the training data, as its predictions differ significantly from the true values, potentially leading to underfitting. In contrast, a model with high variance tends to produce large fluctuations in predictions when small changes occur in the training data, which may result in overfitting. Choosing between estimators reflects a trade-off between bias and variance, aiming for the optimal capacity. Figure 2.6 demonstrates the relationship between the bias-variance trade-off and model optimization.
In terms of outlier detection problems, reducing bias is generally more difficult than reducing variance because ground-truth labels are often unavailable for correcting errors during model training [1]. Ensemble learning can be considered an approach to optimizing the bias-variance trade-off through combining different models that reduce either bias or variance.

Model Combination for Outlier Ensembles

Base models in an ensemble may consist of independent algorithms that produce scores on different scales. As a result, directly using the absolute outlier scores from these base models to interpret the degree of outlierness in the ensemble is not appropriate. For instance, the Local Outlier Factor [11] and the k-nearest neighbor algorithm yield scores on different scales. Therefore, outlier scores generated by each base detector are often normalized before the combination step using common normalization techniques, such as Min-Max or Z-score [56], or are converted into probability estimates [23]. Alternatively, raw scores can be combined with rank-based scores to exploit the robustness of ranking methods against the instability of raw scores, since ranks preserve the relative ordering of anomalies among data samples.

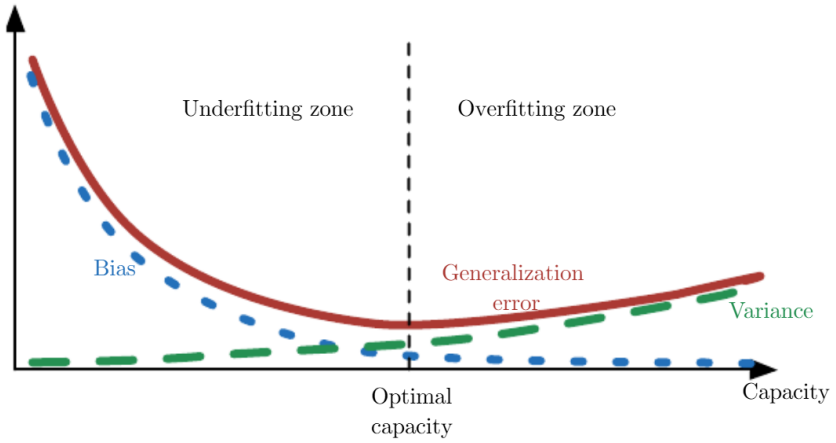


Figure 2.6: Illustration of the bias-variance trade-off and model optimization [27]. This relationship is similar to the relationship between overfitting, underfitting, and model capacity. Increasing model capacity may lead to a decrease in bias, but an increase in variance, resulting in a different curve for generalization error. Therefore, finding an optimal capacity is important.

The final step in an ensemble-based approach is to integrate multiple outlier scores from various base models into a unified score for each data sample. Commonly used combination functions include averaging, median, and maximum. Different combination methods may lead to varying ensemble performance due to their influence on bias and variance reduction. Moreover, it has been shown that the effectiveness of combination functions may vary across different settings [1]. Averaging and median-based aggregation are generally considered relatively stable because of their ability to reduce variance, thereby improving the overall accuracy of the ensemble. In contrast, the maximization function is less stable, as it may reduce bias but can also increase variance. However, the maximization function may perform better when combined with rank-based scores.

2.6 Evaluation Methods

The outlier detection problem can be formulated as a binary classification task that separates data into two classes: inliers and outliers. Therefore, evaluation metrics commonly used for binary classification can also be applied to assess the performance of outlier detection models. In this section, we briefly introduce three common evaluation methods frequently used in the outlier detection literature.

2.6.1 Area under ROC curve

The receiver operating characteristic (ROC) curve is the advanced and widely used measure in evaluating outlier detection models, plotting the true positive rate (TPR) against the false positive rate (FPR). A ROC curve visualizes the trade-off between the proportion at

which the model accurately detects true outliers and the rate at which it mislabels inliers as outliers [31]. Because it is difficult to evaluate the plot directly, the resulting monotone curve is turned into a measure by computing the area under ROC curve (AUROC) [82]. The AUROC serves as a measure of model accuracy, allowing to compare the performances of multiple models numerically. While an AUROC value of 1 indicates a perfectly accurate model, an AUROC value of 0.5 reveals that the model is not capable of distinguish between inlier and outlier. Figure 2.7 shows how ROC curves are plotted in a diagram as well as a detail explain of related aspects.

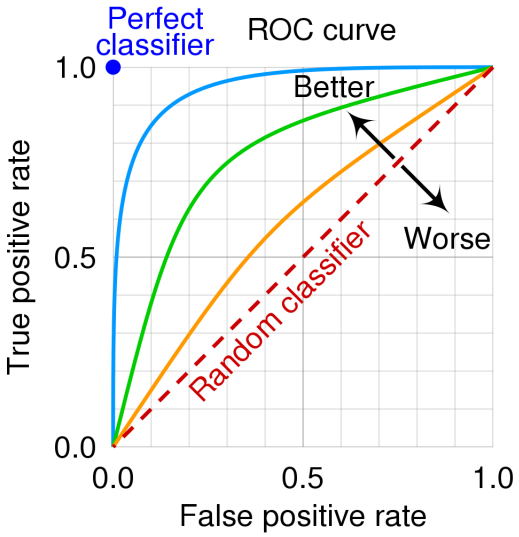
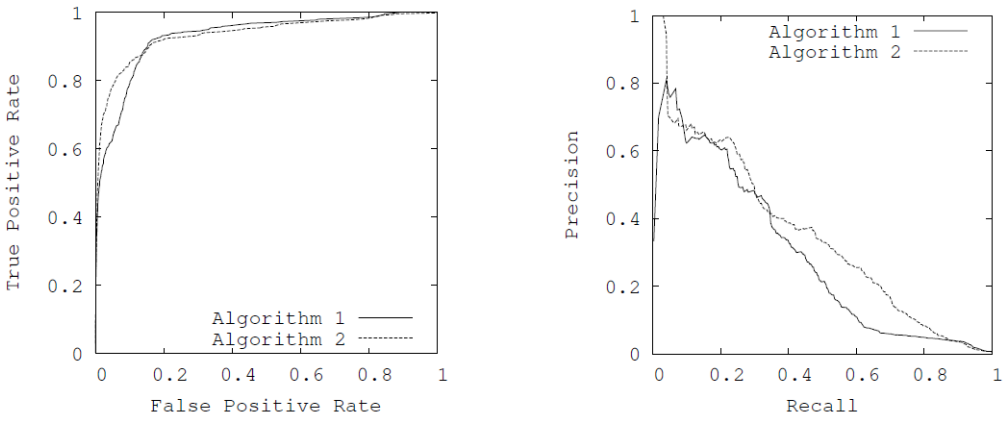


Figure 2.7: The ROC curves of three different models [72]. The diagonal line represents a random guessing model. The closer a model's ROC curve is to the diagonal line, the less accurate it is. Accordingly, the model corresponding to the blue curve performs best, whereas the one corresponding to the orange curve performs worst.

2.6.2 Area under Precision-Recall curve

ROC curves are commonly used to assess results for binary decision problems in machine learning, especially in outlier detection, where datasets are typically imbalanced. However, when dealing with highly imbalanced datasets, which is often the case in outlier detection, Precision-Recall (PR) curves are a useful alternative to ROC curves, as they can highlight performance differences that may not be visible in ROC curves [25]. A crucial difference between ROC and PR curves lies in their visual representation, as illustrated in figure 2.8, which shows ROC and PR curves obtained from the same learned models on a highly imbalanced cancer detection dataset [16]. In the ROC curve, the optimal performance is located in the upper-left corner, whereas in the PR curve, it is located in the upper-right corner. In ROC space, the performances of the algorithms seem to be comparable, but the algorithm 2 shows a clearly better performance over the algorithm 1 in PR space.

The reason for this difference is that the number of negative examples greatly surpasses that of positive examples. Consequently, even a substantial change in the number of false positives results in only a small shift in the false positive rate used in ROC analysis [16]. Meanwhile, precision captures the impact of the large number of negative examples on the algorithm's performance. Similar to ROC curves, the information contained in a PR curve can be summarized by computing the area under the PR curve (AUCPR). Generally, a higher AUCPR score indicates better model performance for a given task.



(a) Comparison in ROC space (b) Comparison in PR space

Figure 2.8: The difference between comparing algorithms in ROC and PR curves [16]

2.6.3 F1 score

An other common approach of evaluating machine learning algorithms is to use Recall, Precision, and F1 score. These metrics were originally developed in the field of Information Retrieval and are now widely used to assess the performance of outlier detection models. In the context of outlier detection, these metrics are based on the counts of true positive (TP), true negative (TN), false positive (FP), and false negative (FN). True positives and false positives denote the number of predicted outliers that are correct and incorrect, respectively. Similarly, true negatives and false negatives represent the number of predicted inliers that are correct and incorrect.

Recall or sensitivity measures the proportion of actual outlier cases that are correctly predicted as outliers [58]:

$$Recall = \frac{TP}{TP + FN} \tag{2.5}$$

Precision or confidence measure the proportion of predicted outlier cases that are actually outliers [58]:

$$Precision = \frac{TP}{TP + FP} \tag{2.6}$$

A higher value of recall or precision alone does not necessarily indicate strong model performance. These metrics focus primarily on the predicted and actual outliers and do not directly reflect how well the model handles inliers. Therefore, a combined evaluation metric such as the F1 score is often used to provide a more balanced assessment. The F1 score is defined as the harmonic mean of precision and recall [14]:

$$F1 \;\; score = \frac{2TP}{2TP + FP + FN} = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{2.7}$$

Because the harmonic mean penalizes large discrepancies between precision and recall, a high F1 score can only be achieved when both metrics are simultaneously high. This property makes the F1 score particularly suitable for evaluating outlier detection models where datasets are highly unbalanced.

Chapter 3

Methodology

This chapter presents the methodology for constructing an ensemble outlier detection model, in which base models extend the FoMo-0D architecture introduced in [63]. To generate diverse synthetic datasets for pre-training of foundation models, we employ not only Gaussian Mixture Models (GMMs), as used in FoMo-0D, but also introduce Modified Gaussian Mixture Models (MGMMs) and Cube Mixture Models (CMMs). With the goal of training FoMo-0D-based foundation models that are computationally efficient while achieving comparable or even enhanced performance relative to the original FoMo-0D model, we utilize various optimization strategies, facilitating the creation of a robust ensemble under resource limitations. Furthermore, we propose three ensemble strategies, referred to as **Bagging**, **Selected Bagging**, and **Weight Average**, to improve the generalization of foundation models in outlier detection.

3.1 A Data Prior for Outlier Detection

The choice of data prior is fundamental to the performance of our foundation models, as they are based on PFNs that learn to approximate the PPD for a given prior. The data prior should reflect distributions as general and diverse as those observed in the real world, so that the trained model can generalize effectively to downstream real-world outlier detection datasets. In this section, we describe the data priors employed to generate diverse synthetic datasets for pre-training the foundation models.

3.1.1 Gaussian Mixture Model

FoMo-0D model employs a data prior based on Gaussian Mixture Models (GMMs), from which numerous outlier detection (OD) datasets are simulated for the pre-training stage. Each synthetic dataset contains an equal proportion of inliers and outliers. These samples are drawn from a specific GMM configuration, where inliers correspond to samples within the 90th percentile of the GMM, while outliers lie outside the 90th percentile. The region

within the 90th percentile of the GMM is referred to as the original GMM, whereas the region outside the 90th percentile is referred to as the inflated GMM.

For each dataset, inlier samples are drawn from the original GMM with m clusters in d-dimension space, with cluster centers $\mu_{jk} \in [-5, 5]$, $j \in [m]$, $k \in [d]$, and *diagonal* $\sum_j$ whose entries lie in $(0, 5]$ [63]. The original GMM is constructed with $m \leq M$ and $d \leq D$ selected uniformly at random, where M denotes the maximum number of clusters and D is the maximum feature dimensionality. To generate outliers, a subset of dimensions H with $|H| \leq d$ is first sampled at random. Outliers are then drawn from the inflated GMM based on their Mahalanobis distances, which shares the same centers μ_{jk} with the original GMM but has the inflated covariances $5 \times \sum_{j,hh}$ for $h \in H$ [63].

Figure 3.1 presents three illustrative synthetic datasets in 2D generated from GMMs containing one, two, and three clusters respectively. In this example, the 80th percentile is used as the decision boundary separating the original from inflated GMM.

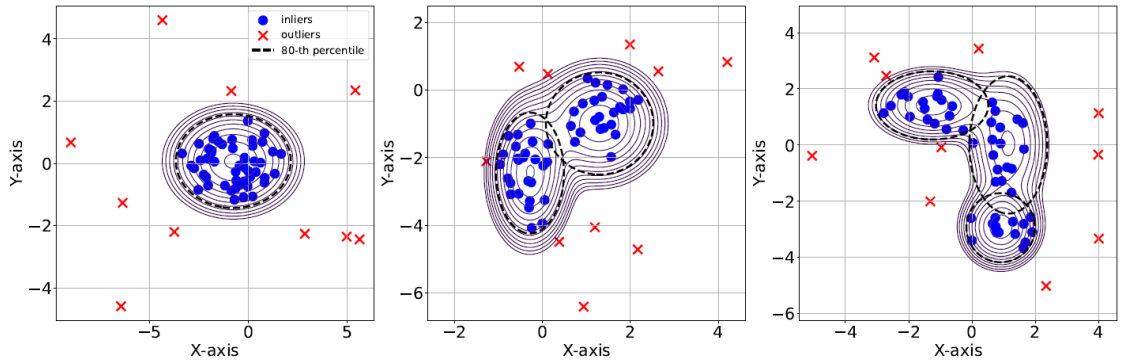


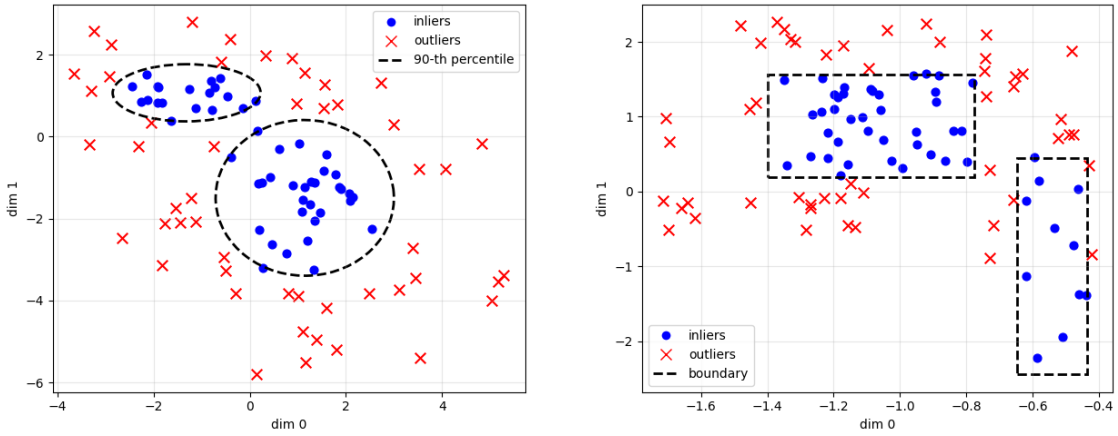
Figure 3.1: Demonstration of synthetic data in 2D drawn from a GMM with 80th percentile as the criterion [63]

3.1.2 Modified Gaussian Mixture Model

In the original FoMo-0D framework, inliers are sampled uniformly across all clusters within a given GMM configuration. However, uniform sampling assumes balanced cluster proportions, which may not accurately reflect real-world data distributions. To enhance the diversity and realism of the synthetic datasets, we extend the GMM prior by introducing non-uniform mixture weights. Specifically, each cluster in a GMM configuration is assigned a randomly generated positive weight, with all weights normalized to sum to one. The sampling probability of inliers from each cluster is then proportional to its assigned weight. Clusters with larger weights contribute more inlier samples, while those with smaller weights contribute fewer samples. This modification allows the synthetic datasets to exhibit varying cluster densities and imbalance patterns, which more closely resemble real-world data distributions. Hence, it improves the prior's expressiveness and its ability

to approximate real-world data-generating processes. We call this alternative version as Modified Gaussian Mixture Model (MGMM).

Figure 3.2(a) demonstrates 50 data points randomly selected from a two-dimensional synthetic dataset generated using an MGMM with two clusters. The 90th percentile is used as the decision boundary separating the original MGMM from the inflated MGMM.



(a) Synthetic data drawn from a MGMM

(b) Synthetic data drawn from CMM

Figure 3.2: Illustration of two-dimensional synthetic datasets drawn from a MGMM with the 90th percentile as the criterion and CMM.

3.1.3 Cube Mixture Model

In addition to GMM and MGMM, we employ another data prior based on Cube Mixture Models (CMMs). Each dataset drawn from a specific CMM configuration contains an equal proportion of inliers and outliers, where inliers correspond to samples located inside the volume of the CMM, and outliers lie outside this region.

An original CMM is constructed with $m \leq M$ and $d \leq D$ selected uniformly at random, where M denotes the maximum number of clusters and D is the maximum feature dimensionality. Each cluster $j \in [m]$ is defined by a center $\mu_j \in \mathbb{R}^d$ and a deviation vector $\delta_j \in \mathbb{R}^d_+$, which determines the half-width of the axis-aligned hyperrectangle associated with the cluster. Thus, each cluster defines a cube region

$$C_j = \{x \in \mathbb{R}^d : \mu_j - \delta_j \leq x \leq \mu_j + \delta_j\} \tag{3.1}$$

For each dimension $k = 1, ..., d$, the coordinate x_k satisfies $\mu_{jk} - \delta_{jk} \leq x_k \leq \mu_{jk} + \delta_{jk}$. The cluster centers are generated randomly with coordinates approximately within the interval $[-5, 5]$. The deviation vectors are generated independently for each dimension by multiplying two uniformly sampled random variables, resulting in positive values bounded by 3. Similar to MGMM, we also assign a randomly generated positive weight to each

cluster, with all weights normalized to sum to one. Inlier samples are then drawn uniformly from the corresponding hyperrectangle. To determine whether a sample lies inside or outside a cluster region, a normalized cubic distance is used. For a sample x and cluster j, the cubic distance is defined as

$$d_j(x) = \max_{k \in H} \frac{|x_k - \mu_{jk}|}{\delta_{jk}}, \tag{3.2}$$

where H ($|H| \leq d$) is a randomly selected subset of dimensions used for anomaly evaluation. A sample x is considered an inlier if its minimum cubic distance to all clusters satisfies $d_j(x) < 1 - \Delta$, where Δ is a small margin parameter with $\Delta = 0.05$ by default.

To generate outliers, candidate samples are drawn from the inflated CMM, which shares the same cluster centers as the original model but expands the deviation vectors by a constant inflation factor of 2. In other words, the inflated cubes are defined by $\mu_j \pm 2 \times \delta_j$. Samples are drawn uniformly from these inflated regions and are retained as outliers only if their cubic distance to every cluster satisfies $d_j(x) > 1 + \Delta$. This procedure ensures that outliers lie outside the original cube regions while remaining relatively close to cluster boundaries, producing local anomalies that are more challenging for anomaly detection models to identify.

Figure 3.2(b) demonstrates 50 data points randomly selected from a two-dimensional synthetic dataset generated using an CMM with two clusters.

3.2 Strategies for Optimization of Foundation Models

Limited infrastructure resources make it challenging to train large foundation models, as they typically contain a vast number of parameters and are computationally expensive. This creates a bottleneck for scaling foundation models, even though their performance often improves with increasing model size. To construct an ensemble of foundation models for outlier detection under infrastructure constraints, we aim to train FoMo-0D-based foundation models that require fewer computational resources for both training and inference, while achieving comparable or even improved performance relative to the original FoMo-0D model.

Our approach for training smaller FoMo-0D-based foundation models is inspired by the concept of Parameter-Efficient Fine-Tuning (PEFT), particularly selective and hybrid PEFT methods. Based on the assumption that certain parameters in pre-trained models are especially important for a specific task, we explore different hyperparameter configurations that reduce infrastructure requirements while maintaining or improving model performance. Similar to selective PEFT methods, our approach involves either fine-tuning only a subset of the existing hyperparameters while freezing the remaining ones, or introducing additional hyperparameters that were not present in the original FoMo-0D configuration. In

addition, we also combine different techniques to leverage their individual strengths, following the idea of the hybrid PEFT methods. Specifically, we are able to conduct experiments covering the following aspects, but not limited to:

- Exploring different data priors used to generate synthetic datasets for pre-training the foundation models. Instead of relying solely on the GMM-based prior employed in the original FoMo-0D model, we also investigate alternative distributions described in the previous section. In this way, the data prior is treated as a tunable hyperparameter.
- Experimenting with different hyperparameter values in the data prior settings. In FoMo-0D, the GMM-based prior is defined with fixed hyperparameters, including the maximum number of clusters $M = 5$, maximum feature dimensionality $D = 100$, and uniform cluster weights. In our approach, these parameters are treated as tunable hyperparameters. Similarly, the hyperparameters in MGMM-based and CMM-based priors are also adjusted in order to optimize model performance.
- Investigating the influence of external training hyperparameters such as the number of epochs, batch size, and learning rate, which can significantly affect the training process of a machine learning model. Increasing the value of one or more hyperparameters may increase the model size or computational cost. However, by combining such increases with reductions in other hyperparameters, it is possible to control the overall model complexity and resource consumption.
- Experimenting with variations in the model architecture by tuning hyperparameters of the transformer used in FoMo-0D. Transformer architectures include many configurable hyperparameters, and one of the main challenges is identifying the most relevant parameters and their appropriate values under the constraints of limited time and infrastructure resources.
- Evaluating different combinations of hyperparameter configurations in order to identify settings that achieve strong model performance while maintaining efficient use of computational resources.

3.3 Ensemble Strategies

The goal of ensemble learning is to combine multiple models in a way that improves the performance and robustness of the final prediction. However, unlike traditional machine learning models, foundation models contain a large number of parameters and are trained on vast amounts of data, requiring substantial computational resources not only during pre-training but also during inference. As a result, constructing ensembles of foundation models becomes more challenging, as it entails significantly higher computational costs and increased inference time.

To construct an ensemble of foundation models for outlier detection under infrastructure constraints, our approach consists of two steps. First, we train FoMo-0D-based foundation models that require fewer computational costs, while obtaining comparable or even improved performance relative to the original FoMo-0D model. This approach is described in Section 3.2. Second, we design an ensemble learning framework that combines diverse foundation models through a unified decision-making mechanism. In this stage, we investigate three different ensemble strategies. Figure 3.3 provides an overview of the overall process.

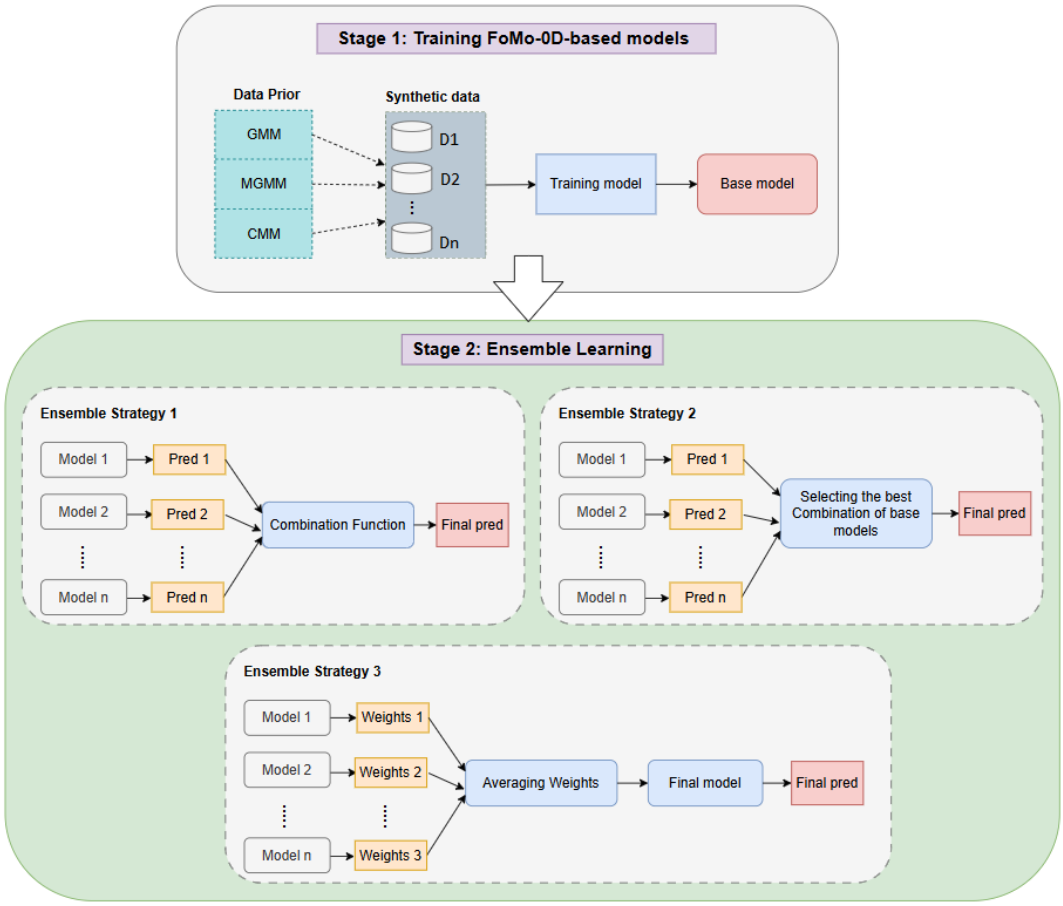


Figure 3.3: Overview of the proposed framework for constructing ensembles of foundation models. In the first stage, multiple base models are trained using the FoMo-0D architecture. In the second stage, these models are combined through a unified decision-making mechanism, where three different ensemble strategies are explored.

3.3.1 Ensemble Strategy One - Bagging

We refer to the first ensemble strategy as **Bagging**, which systematically aggregates predictions from multiple base models. In this framework, each sub-model f_m is trained independently using the FoMo-0D architecture. The sub-models may differ in their ar-

chitectural configurations and may be trained on datasets generated from different data priors, ensuring diversity among components in the ensemble.
After each base model f_m has been trained, we apply it to infer outlier scores for all data samples. Each sub-model computes the outlier score as the probability of the anomaly class in a binary classification setting. Specifically, each dataset contains two classes $c \in \{0, 1\}$, where 0 denotes normal instances and 1 denotes outliers. For each instance i, the base model f_m produces a vector of logits over two classes, denoted as $[z_{i,0}, z_{i,1}]$, where $z_{i,0}$ and $z_{i,1}$ correspond to the normal and anomaly classes, respectively. These logits are transformed into probabilities using the softmax function:

$$p_{i,c} = \frac{e^{z_{i,c}}}{e^{z_{i,0}} + e^{z_{i,1}}} \tag{3.3}$$

The outlier score assigned to the instance i by model f_m is defined as $s_i^{(m)} = p_{i,1}$. In this strategy, all sub-models contribute equally to the final prediction. Given M trained base models $(f_1, ..., f_M)$, the final ensemble score for each instance i is computed as the average of the anomaly probabilities across all models:

$$\hat{s}_i = \frac{1}{M} \sum_{m=1}^{M} s_i^{(m)} \tag{3.4}$$

3.3.2 Ensemble Strategy Two - Selected Bagging

The second ensemble strategy extends the first approach and is referred to as **Selected Bagging**. Instead of averaging the predictions of all base models as in the first strategy, this framework evaluates different combinations of sub-models and selects the subset that achieves the best overall performance across all datasets D. The final predictions are then obtained by averaging the outlier scores of only the selected sub-models.
In particular, given M trained sub-models $(f_1, ..., f_M)$, for every candidate subset of models $E \subseteq \{2, ..., M\}$, the ensemble score for each data instance i is computed as:

$$\overline{s}_i^{(E)} = \frac{1}{|E|} \sum_{m \in E} s_i^{(m)} \tag{3.5}$$

To evaluate the performance of various candidate subsets, we use the average area under ROC curve (AUROC) across all datasets D as the evaluation metric. The subset that achieves the highest mean AUROC is selected as the optimal ensemble:

$$E^* = \arg\max_E \frac{1}{D} \sum_{d \in D} AUROC_d(E) \tag{3.6}$$

Once the optimal subset E^* is identified, the final outlier score for each instance i is calculated by averaging the outlier scores of the selected models:

$$\hat{s}_i = \frac{1}{|E^*|} \sum_{m \in E^*} s_i^{(m)} \tag{3.7}$$

Since the Selected Bagging strategy evaluates all candidate subsets of base models, the running time of the model combination step grows exponentially with the number of sub-models, namely $\mathcal{O}(2^M)$. In contrast, the computational complexity of model combination in the Bagging strategy increases linearly with the number of base models, namely $\mathcal{O}(M)$. However, once the optimal subset E^* has been identified, the computational complexity in the inference phase of Selected Bagging strategy is reduced to $\mathcal{O}(N \times |E^*| \times T)$, with $|E^*| \leq M$, N is the number of data instances, and T denotes the inference cost of a single base model. In comparison, the overall computational cost in inference phase of the Bagging strategy is $\mathcal{O}(N \times M \times T)$.

The difference in inference complexity between the two strategies particularly evident when an optimal E^*, consisting of only a few sub-models, outperforms the full set of M base models. The Selected Bagging strategy is based on the principle that carefully selecting a subset of the most suitable models can be more effective than including all available candidates in the ensemble.

3.3.3 Ensemble Strategy Three - Weight Average

The first two ensemble strategies that we introduced above follow a widely adopted paradigm in machine learning, where multiple models are trained and their predictions are subsequently aggregated. Nevertheless, this approach can become infeasible under memory and computational constraints, as the inference phase requires storing multiple models and performing a forward pass through each of them. This limitation is further amplified in the context of foundation models, which demand substantial resources for both pre-training and inference. To address this challenge, we investigate an alternative ensemble strategy, referred to as **Weight Average**, inspired by [66, 75].

In this strategy, instead of waiting for all base models to infer outlier scores for all data samples and then combining these scores, we first average the weights of the sub-models to construct the final model. The final model is then used to infer outlier scores for all data samples. As a result, the cost of the model combination step is reduced dramatically to $\mathcal{O}(1)$, and the overall inference complexity is decreased to $\mathcal{O}(N \times T)$, leading to a more computationally efficient solution.

Although the weight average strategy is much less expensive than the other ensemble strategies, its preconditions are relatively strict. It is only possible to directly average the weights of different models in special cases, such as when they share the same architecture [43], or when a model is fine-tuned on similar data for a small number of steps [75]. This means that the final model can leverage the strengths of multiple models when their weights are sufficiently similar but still exhibit some differences. The vanilla averaging [66] has been shown to perform poorly when applied to trained models whose weights differ substantially.

In the next section, we conduct experiments covering the following cases:

- Ensembles of similar models that differ in specific aspects, such as data priors, training epochs, batch sizes, and other hyperparameters.
- Ensembles constructed from multiple checkpoints of models sharing the same architecture and portions of the training trajectory.

Chapter 4

Experiments and Evaluation

This chapter presents various experiments conducted to investigate the performance of different ensemble strategies of foundation models for outlier detection. In particular, we aim to address the following research questions: How can FoMo-0D-based foundation models be optimized to achieve performance comparable to or exceeding that of the original FoMo-0D under constrained computational resources? Which ensemble strategies are most effective in enhancing the generalization capability of foundation models for outlier detection?

4.1 Experiment Setup

In this section, we describe the benchmark datasets used to evaluate the performance of our outlier detectors. We then discuss the baseline methods, as well as the hyperparameters considered for optimization to obtain base models that perform comparably to the original FoMo-0D model while requiring fewer computational resources.

4.1.1 Benchmark Datasets

We conduct experiments on 57 tabular outlier detection datasets from the ADBench [32] benchmark. This benchmark comprise a set of 47 popular datasets covering various domains and scales, as well as 10 newly constructed tabular datasets derived from image and natural language tasks, where pre-trained models are employed to extract embeddings. A detailed description of these datasets is provided in Table 4.1.
To ensure a fair comparison with FoMo-0D [63] and its corresponding baselines, we adopt their outlier detection setting. Specifically, the training set D_{train} contains only inlier samples, while the test set D_{test} consists of the remaining inliers and all outlier samples. However, we do not follow the idea of training one model with five different seeds due to computational resource limitations, therefore we split each dataset only one time into train/test using a fixed seed of 0. This seed is consistently used across all experiments. As in FoMo-0D, the training set D_{train} is used solely as context for inference.

Table 4.1: Description of all tabular outlier detection datasets in ADBench [32]. The last ten datasets in the table are image and text datasets that are vectorized through pre-trained ResNet18 [36] and BERT [45] models, respectively.

Dataset Name	#Samples	#Features	#Anomaly	%Anomaly	Category
ALOI	49534	27	1508	3.04	Image
annthyroid	7200	6	534	7.42	Healthcare
backdoor	95329	196	2329	2.44	Network
breastw	683	9	239	34.99	Healthcare
campaign	41188	62	4640	11.27	Finance
cardio	1831	21	176	9.61	Healthcare
Cardiotocography	2114	21	466	22.04	Healthcare
celeba	202599	39	4547	2.24	Image
census	299285	500	18568	6.20	Sociology
cover	286048	10	2747	0.96	Botany
donors	619326	10	36710	5.93	Sociology
fault	1941	27	673	34.67	Physical
fraud	284807	29	492	0.17	Finance
glass	214	7	9	4.21	Forensic
Hepatitis	80	19	13	16.25	Healthcare
http	567498	3	2211	0.39	Web
InternetAds	1966	1555	368	18.72	Image
Ionosphere	351	32	126	35.90	Oryctognosy
landsat	6435	36	1333	20.71	Astronautics
letter	1600	32	100	6.25	Image
Lymphography	148	18	6	4.05	Healthcare
magic.gamma	19020	10	6688	35.16	Physical
mammography	11183	6	260	2.32	Healthcare
mnist	7603	100	700	9.21	Image
musk	3062	166	97	3.17	Chemistry
optdigits	5216	64	150	2.88	Image
PageBlocks	5393	10	510	9.46	Document
pendigits	6870	16	156	2.27	Image
Pima	768	8	268	34.90	Healthcare
satellite	6435	36	2036	31.64	Astronautics
satimage-2	5803	36	71	1.22	Astronautics
shuttle	49097	9	3511	7.15	Astronautics
skin	245057	3	50859	20.75	Image
smtp	95156	3	30	0.03	Web
SpamBase	4207	57	1679	39.91	Document
speech	3686	400	61	1.65	Linguistics
Stamps	340	9	31	9.12	Document
thyroid	3772	6	93	2.47	Healthcare
vertebral	240	6	30	12.50	Biology
vowels	1456	12	50	3.43	Linguistics
Waveform	3443	21	100	2.90	Physics
WBC	223	9	10	4.48	Healthcare
WDBC	367	30	10	2.72	Healthcare
Wilt	4819	5	257	5.33	Botany
wine	129	13	10	7.75	Chemistry
WPBC	198	33	47	23.74	Healthcare
yeast	1484	8	507	34.16	Biology
CIFAR10	5263	512	263	5.00	Image
FashionMNIST	6315	512	315	5.00	Image
MNIST-C	10000	512	500	5.00	Image
MVTec-AD	5354	512	1258	23.50	Image
SVHN	5208	512	260	5.00	Image
Agnews	10000	768	500	5.00	NLP
Amazon	10000	768	500	5.00	NLP
Imdb	10000	768	500	5.00	NLP
Yelp	10000	768	500	5.00	NLP
20newsgroups	11905	768	591	4.96	NLP

4.1.2 Baseline Methods

We compare against all 27 baselines, including the FoMo-0D model [63] and its 26 corresponding baselines. The baselines used in [63] are imported from the state-of-the-art diffusion-based model DTE [53] and its variants: DTE-C, DTE-IG, and DTE-NP. We employ the pre-trained FoMo-0D model with $D = 100$, available at `https://github.com/A-Chicharito-S/FoMo-0D/blob/main/ckpt.zip`. The remaining baselines include classical methods: CBLOF [37], COPOD [50], ECOD [51], FeatureBagging [49], HBOS [26], IForest [52], kNN [48], LODA [57], LOF [11], MCD [20], OCSVM [62], and PCA [65]. The deep learning models include DeepSVDD [61], DAGMM [83], DROCC [29], GOAD [7], ICL [64], SLAD [78], and DIF [77]. In addition, we include four generative baselines: normalizing planar flows [59], DDPM [38], VAE [47], and GANomaly [3].
Due to computational resource constraints, we cannot retrain all the baselines. Therefore, we use the results reported in [53] and [63] for comparison in our experiments. Besides comparing our models with FoMo-0D and 26 other baselines, we also focus on comparison with FoMo-0D and the top four best-performing baselines (in order: DTE-NP, kNN, ICL, and DTE-C) as reported in [63].

4.1.3 Choice of Hyperparameters for Optimization

Data priors are used to generate synthetic datasets for pre-training base models. In this work, we consider three types of data priors, as introduced in section 3.1: GMMs, MGMMs, and CMMs. Beyond conducting experiments with different data priors, we systematically tune a range of hyperparameters to obtain base models that achieve strong performance while ensuring computational efficiency. Although the original FoMo-0D framework involves many different hyperparameters, we focus on a selected subset that is most relevant to our experiments. In the following, we briefly describe the hyperparameters considered for optimization:

- **max_feature_dim**: The maximum number of features that a synthetic training dataset can have. Larger feature dimensions increase computational cost and memory requirements during training.

- **max_num_cluster**: The maximum number of clusters in a synthetic dataset, which controls the complexity of the data distribution.

- **batch_size**: In the FoMo-0D setting, the batch size represents the number of training datasets passed through a model in one iteration. While using a large batch may result in the poor generalization performance of the model and the risk of overfitting [46], too small batches take a long time to converge, requiring more training time but less memory.

- **epochs**: Epochs denote the number of times that a model works through the whole training dataset. A small number of epochs may lead to underfitting, as the model may not have enough training time to learn complex patterns in the data. However, a large number of epochs increases computational cost and may result in overfitting, as the model may memorize the training data and perform poorly on new, unseen data.

- **lr**: Learning rate controls how much a model adjusts its weights at each iteration in response to the error encountered. A low learning rate can improve the generalization of a model but increases computation time because of slow convergence. Meanwhile, a high learning rate speeds up training but may lead to divergence of the loss function.

- **nlayer**: The number of layers in the transformer encoder of the FoMo-0D architecture. Since each layer contains weights and biases that need to be stored in memory, increasing the amount of layers leads to more parameters. Consequently, it requires more computational cost and memory usage during both training and inference phases.

- **nhead**: The number of attention heads in the transformer encoder block. A higher number of attention heads requires significantly more memory consumption because of the quadratic complexity of computing attention scores over input sequences.

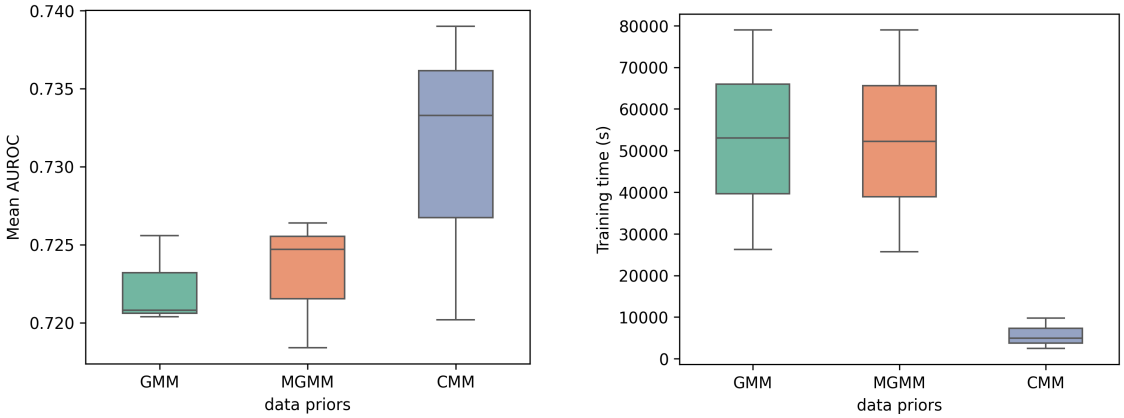
4.2 Experiment Results

4.2.1 Performance of Base Models

As a part of our ensemble approach, we aim to train FoMo-0D-based foundation models that require fewer computational resources for both training and inference, while achieving comparable or even improved performance relative to the original FoMo-0D model. This section presents the remarkable evaluation results obtained during the optimization step.

4.2.1.1 Impact of Data Priors on Performance

To investigate the effect of different data priors on model performance, we train base models on three groups of synthetic datasets generated using GMM, MGMM, and CMM priors, respectively. In total, we have nine base models, categorized into three groups according to the underlying data prior. To reduce model complexity and computational cost for this experiment, the base models within each group differ only in the number of epochs, specifically 10, 20, and 30 epochs. Figure 4.1 illustrates the performance of these model groups in terms of mean AUROC against the ADBench benchmark, and their training time (in seconds).



(a) Mean AUROC scores of each model group

(b) Training time of each model group.

Figure 4.1: The influence of data priors on base models in terms of performance and training time. Figure (a) demonstrates the mean AUROC scores of three model groups categorized by their data priors. Figure (b) presents the model training time (in seconds) of each group.

From Figure 4.1(a), we observe that most base models in the CMM prior group achieve higher performance compared to those in the other groups, followed by models in the MGMM prior group. Although models in the GMM group generally exhibit lower performance, their results are more stable under changes of the epoch hyperparameter, as indicated by a smaller variance. In contrast, models in the MGMM and especially the CMM groups show greater variability in performance. In terms of training time, models in the CMM group require substantially less time than those in the GMM and MGMM groups, indicating that the CMM prior contributes significantly to reducing computational cost during pre-training of base models. This efficiency enables the training of more complex base models under limited computational resources.

Overall, the impact of data priors on model performance is moderate rather than dramatic. Therefore, we still consider using all three types of data priors to create a diversity of detectors in an ensemble. However, due to the constraints of computational resources, submodels trained on CMM-based synthetic datasets are more frequently used in subsequent experiments.

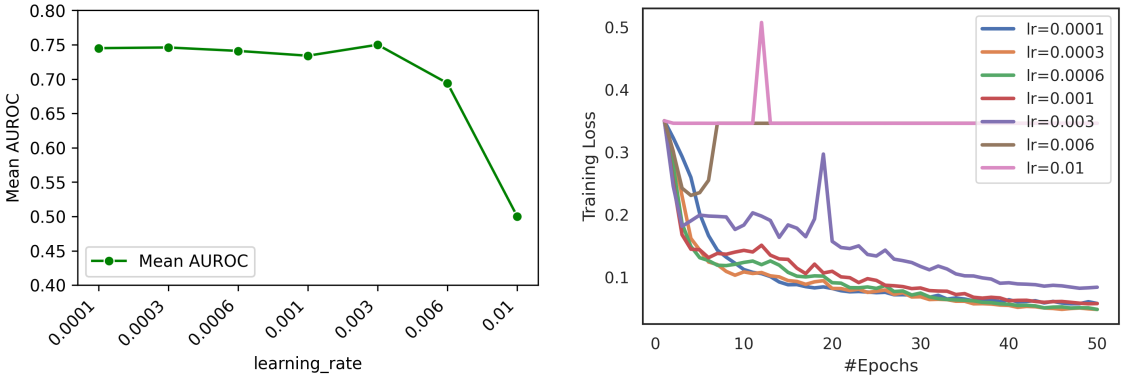
4.2.1.2 Should we optimize max_feature_dim and max_num_cluster?

In the FoMo-0D setting, **max_feature_dim** hyperparameter specifies the maximum number of features that a synthetic training dataset can have, while **max_num_cluster** defines the data distribution complexity of a synthetic dataset. Both parameters play an important role in generating diverse synthetic datasets, thereby promoting diversity among base models within the ensemble. Optimizing these hyperparameters is computationally expensive due to their large search space, and their direct impact on ensemble performance

remains uncertain. An ensemble can benefit from multiple sub-models that are trained on datasets with different dimensional scopes, because each model can learn different parts of the data, generating a more robust final prediction. Therefore, we do not attempt to optimize these hyperparameters but instead use them as factors for creating diversity among sub-models in an ensemble.

4.2.1.3 Optimization of Learning Rate

To identify an appropriate learning rate, we conduct experiments using seven base models that share identical configurations, differing only in their learning rates. The learning rates are varied within the range of 0.0001 to 0.01. Figure 4.2 demonstrates the effect of the learning rate on model performance, as well as the training loss curves of the base models, where each model is labeled according to its learning rate value.



(a) Mean AUROC score of each model

(b) Training loss of each model.

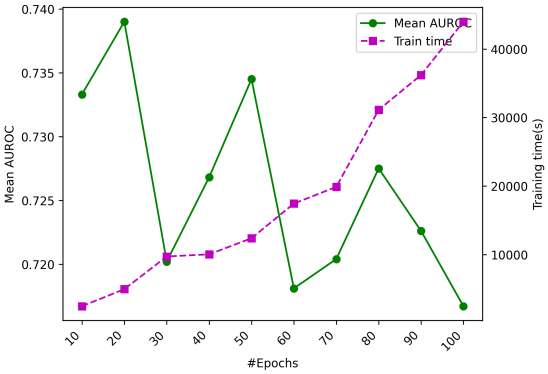
Figure 4.2: The behavior of the learning rate. Figure (a) illustrates the relationship between learning rate and mean AUROC. Figure (b) shows the training loss curves of the models, highlighting their convergence behavior under different learning rates.

Besides considering the influence of the learning rate on the mean AUROC score, the training loss curve also provides useful insights into the choice of learning rate. A highly fluctuating loss curve typically indicates that the learning rate is too big, whereas a stable curve that decreases very slowly suggests that the learning rate is too small [42].
Figure 4.2(a) shows that base models trained with learning rates of 0.006 and 0.01 achieve the lowest mean AUROC scores. Their corresponding training loss curves in Figure 4.2(b) indicate a lack of convergence, suggesting that these learning rates are too high. The base model with a learning rate of 0.003 obtains the highest mean AUROC score. However, its training loss curve in Figure 4.2(b) fluctuates highly, implying that the learning rate may still be too high. In contrast, models trained with learning rates between 0.0001 and 0.001 show stable and steadily decreasing loss curves, while maintaining competitive AUROC performance. Considering additionally that the FoMo-0D architecture incorporates

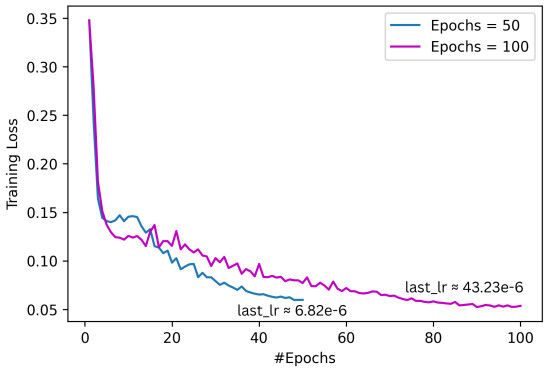
a learning rate scheduler that automatically adjusts the learning rate during training, we select 0.001 as the initial learning rate.

4.2.1.4 How many Epochs are enough?

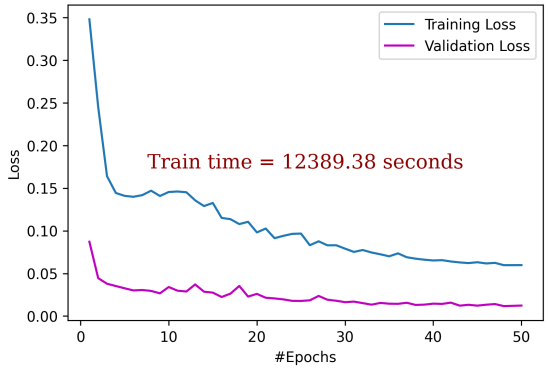
Selecting an appropriate number of epochs is a crucial part of our optimization strategy, as we base our experiments on a NVIDIA RTX 4500 Ada Generation GPU With 24GB of GDDR6 memory, which constraints us from training really large-scale foundation models. Moreover, increasing the number of epochs not only raises computational cost but also may result in overfitting. To determine a reasonal number of epochs, we train ten base models that share identical configurations except for their number of epochs. The number of epochs varies within the range of 10 to 100.



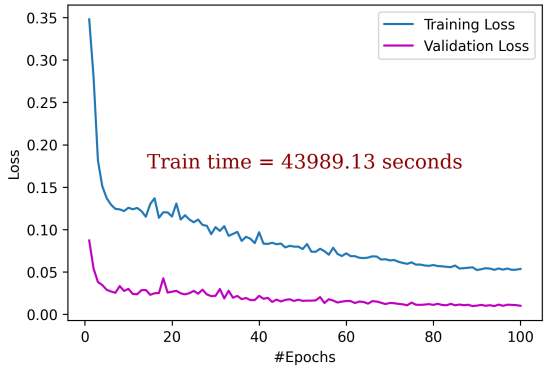
(a) Mean AUROC scores and Training time.



(b) Training losses and the final learning rates of two models.



(c) Training and validation losses of the model with 50 epochs.



(d) Training and validation losses of the model with 100 epochs.

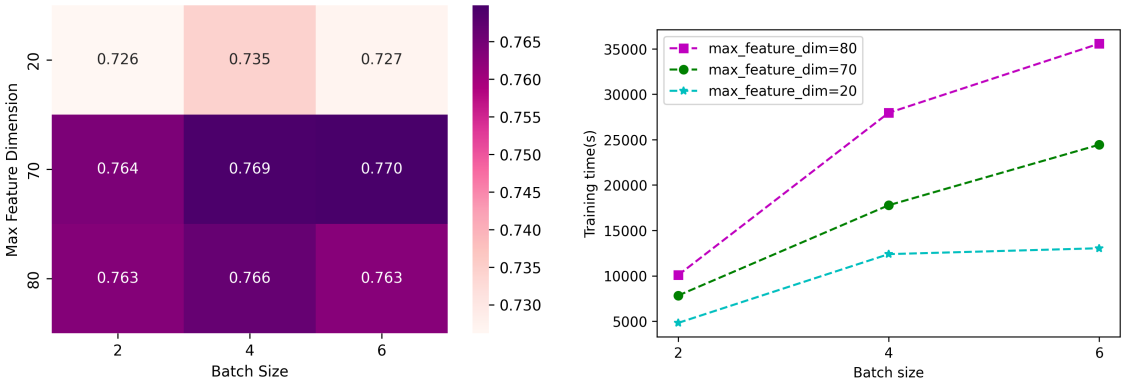
Figure 4.3: The influence of the number of epochs on base models. Figure (a) shows how mean AUROC and training time shift when increasing epochs. Figure (b) displays a comparison of training losses between the base models with 50 and 100 epochs, respectively. It also provides the learning rates at the end of the pre-training phase for both models. Figure (c) and (d) illustrate Training and validation losses of the model with 50 and 100 epochs, respectively.

Figure 4.3(a) demonstrates that increasing the number of epochs leads to a proportional rise in computational cost, as reflected by longer training times. However, the mean AUROC scores of the base models do not increase steadily but instead fluctuate as the number of epochs increases. The base model with 100 epochs performs even worse than all other competitors. To further investigate this behavior, we visualize the training losses and the adjusted learning rates at the end of the pre-training phase for two base models with 50 and 100 epochs, as depicted in Figure 4.3(b). The training loss of the base model with 100 epochs decreases more slowly and is less smooth than that of its competitor, indicating that the model with 50 epochs learns faster and more consistently. We suppose that one of the factors triggering this phenomenon might be the learning rate scheduler integrated into the FoMo-0D architecture. Although both models are initialized with a learning rate of 0.001, their learning rates behave differently during training. We observe that the model with 100 epochs ends up with a higher learning rate value than its competitor. However, we do not investigate this issue further and leave it for future work.

Figure 4.3(c) and (d) provide further insight for determining an appropriate number of training epochs. For the model trained with 50 epochs, both the training and validation losses stabilize around epoch 45, whereas the model trained with 100 epochs reaches its optimal state around epoch 80. This means that beyond these points, further training does not lead to significant improvements, as the models have learned as much as they can from the training data and reached their best performance. Moreover, the 100-epoch model requires more than three times the training time of the 50-epoch model. Taking these factors into account, we select 50 epochs (or fewer) for training base models. Additionally, under limited computational resources, the number of epochs may be reduced further when other hyperparameters are increased.

4.2.1.5 Which batch size is reasonable?

In the FoMo-0D setting, the batch size represents the number of training datasets passed through a model in one iteration, and the number of iterations per epoch is fixed at 1000. That means the number of synthetic datasets used for training a model is proportional to the batch size hyperparameter. Increasing the batch size results in the generation of more synthetic datasets, which are then used in the pre-training process, requiring additional computational cost and memory. Due to memory limitations, We often failed to train base models with a batch size of 8. As a result, our experiments are restricted to batch sizes of 2, 4, and 6. To assess the impact of batch size on model performance, we train nine base models, categorized into three groups according to their **max_feature_dim** hyperparameter values. The base models within each group differ only in the batch size, specifically 2, 4, and 6.



(a) Mean AUROC scores of models across two hyperparameters.

(b) Training time (s) of each model in correlation with batch size.

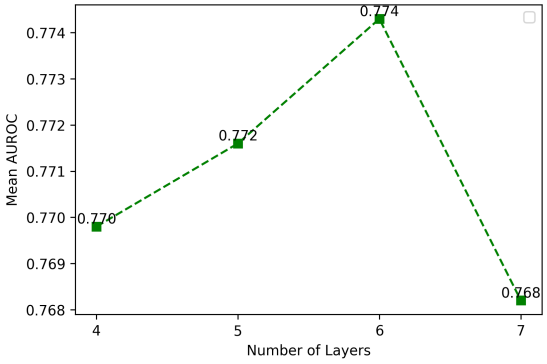
Figure 4.4: The impact of batch sizes on model performance and training time. Figure (a) illustrates mean AUROC scores for varying values of max feature dimension and batch size. Figure (b) highlights that increasing the batch size leads to an increase in the model training time.

This experiment indicates that there is no single optimal batch size value that works for all base models with different configurations. However, batch sizes of 4 or 6 tend to yield better model performance, as depicted in Figure 4.4(a). Although models with higher batch sizes require more computational cost and memory, the increase is not dramatic. Hence, for the purpose of ensemble strategies, we still train base models with both batch sizes of 4 and 6 flexibly, depending on the other hyperparameter configurations of each specific model.

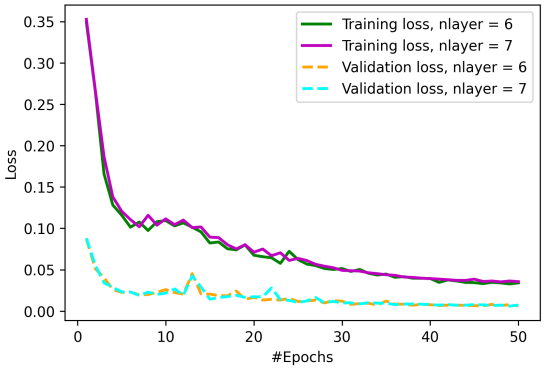
4.2.1.6 Impact of Layers and Attention Heads

So far, the base models introduced in previous experiments are trained with the number of transformer layers (**nlayer**) of 4 and the number of attention heads (**nhead**) of 4, following the original FoMo-0D configuration. To evaluate the impact of these hyperparameters on model performance, we select the best-performing base model and retrain it with varying values of **nlayer** and **nhead**.

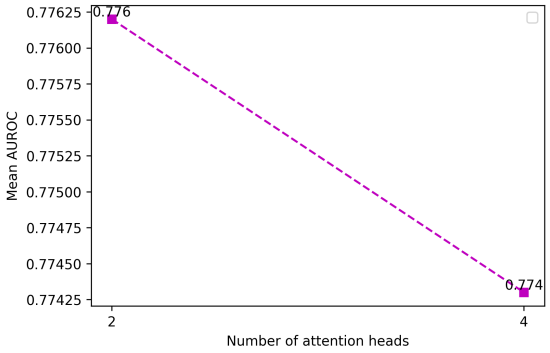
In the first experiment, nhead is fixed at 4, while nlayer is varied from 4 to 7. Figure 4.5(a) shows mean AUROC scores of the model when being trained with different number of layers. The model performance improves as nlayer increases, reaching a peak at nlayer = 6, before dropping significantly at nlayer = 7. This trend suggests that increasing model depth initially enables the model to capture more complex patterns in the data. However, beyond six layers, the additional capacity does not translate into improved performance. Due to hardware constraints, we cannot continue to experiment with nlayer = 8 or further. Notably, Figure 4.5(b) illustrates that the models with nlayer of 6 and 7 have nearly identical training and validation loss curves. Despite this, their performance on new, unseen



(a) Mean AUROC scores of models with various nlayer settings.



(b) Training and validation lossess of two models with nlayer of 6 and 7 respectively.



(c) Mean AUROC scores of models with nhead of 2 and 4.

Figure 4.5: Illustrate the impact of nlayer and nhead hyperparameters. Figure (a) indicates the effect of increasing the number of layers in the transformer encoder block on model performance. Figure (b) demonstrates that two models with nlayer of 6 and 7 have nearly identical training loss curves and validation loss curves. Figure (c) shows that model performance increases when the number of attention heads is reduced.

data differs, with the deeper model performing worse. This indicates that the degradation is not due to classical overfitting, as no divergence between training and validation loss is observed. Instead, it suggests that increasing depth beyond a certain point may lead to poorer generalization, potentially due to optimization instability or the model converging to less robust representations.

In the second execution, we keep the nlayer hyperparameter at 6 and train the model using nhead values of 2 and 4. Attention heads determine how a model focuses on different parts of the input sequence simultaneously to identify which part are more or less important. An interesting observation is that reducing the number of attention heads from 4 to 2 leads to a performance improvement, as shown in Figure 4.5(c). This finding suggests that simply adding more attention heads does not necessarily capture better information. Overall,

the configuration with 6 layers and 2 attention heads achieves the most favorable trade-off between model complexity and performance, and attains the highest mean AUROC among all FoMo-0D-based models that we have trained.

4.2.1.7 Does our base model performs better than the original FoMo-0D?

In this part, we compare the performance of our best base model with the original FoMo-0D. Due to infrastructure resource constraints, we cannot retrain the original FoMo-0D. Therefore, we utilize the model published at `https://github.com/A-Chicharito-S/FoMo-0D/blob/main/ckpt.zip` to evaluate it on the ADBench benchmark. While the original FoMo-0D model employs a GMM-based data prior, our best base model is built upon a CMM-based data prior. The corresponding hyperparameter configurations of both models are summarized in Table 4.2

Table 4.2: The hyperparameter settings of the original FoMo-0D model and our best base model

Hyperparameter	**FoMo-0D**	**Our best base model**
max_feature_dim	100	70
max_num_cluster	5	70
batch_size	8	6
epochs	200	50
lr	0.001	0.001
nlayer	4	6
nhead	4	2

Figure 4.6 presents a comparison of performance and inference time between our best-performing base model and the original FoMo-0D model. For reference, additional points representing other base models ranked within the top 10 are also included. The results show that not only the best base model, but all top-performing models consistently outperform the original FoMo-0D in terms of AUROC. Notably, compared to FoMo-0D, all our base models require significantly less inference time, which is a crucial criterion in real-world scenarios where most applications need to detect anomalies in real time.

To evaluate the effectiveness of our best base model relative to FoMo-0D on complex, high-dimensional data, we analyze their mean AUROC scores across ten challenging datasets from the computer vision (CV) and natural language processing (NLP) domains within the ADBench benchmark. As shown in Figure 4.7, our model demonstrates strong performance on the majority of datasets, except for CIFAR10, FashionMNIST, and SVHN datasets from CV area.

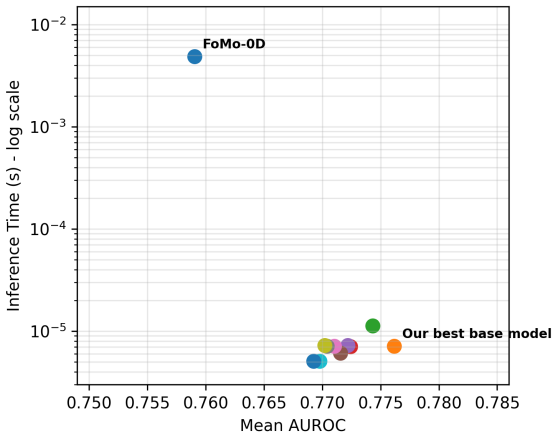
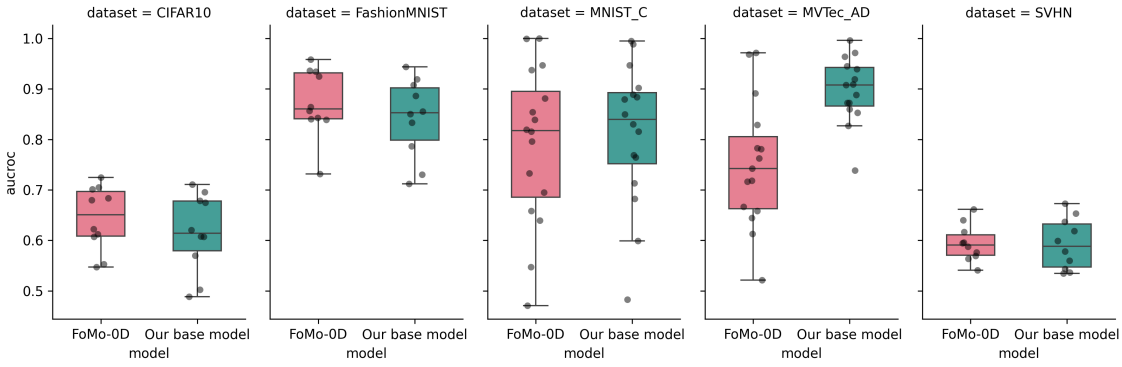
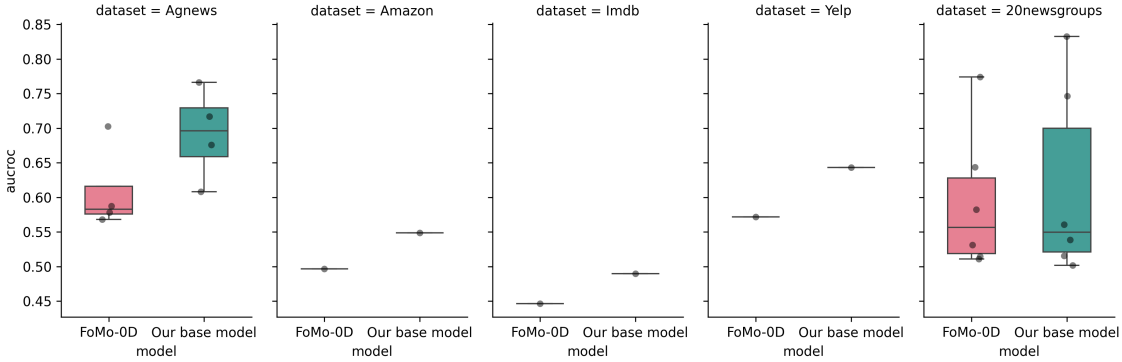


Figure 4.6: Average inference time vs. mean AUROC scores of the original FoMo-0D model (blue) and our best base model (orange) for all 57 ADBench dtatasets. Additional points correspond to other base models ranked among the top 10 in performance.



(a) Mean AUROC comparison across datasets from CV domain.



(b) Mean AUROC comparison across datasets from NLP domain.

Figure 4.7: Mean AUROC comparison between the FoMo-0D and our best base model across ten complex datasets from CV and NLP domains in the ADBench benchmark. For each dataset, results are reported per class, with each point representing the mean AUROC of a single class and boxplots summarizing the distribution across all classes.

Generally, the experimental results demonstrate that it is possible to train outperforming foundation models under infrastructure resource limitations by reasonably optimizing their hyperparameters. In the following section, we further explore the performance of ensemble strategies.

4.2.2 Performance of Ensemble Strategies

4.2.2.1 Which combination function is best?

Figure 4.8 illustrates the performance of three ensembles with 5, 6, and 7 base models on the ADBench benchmark. Generally, when aggregating outlier scores from base models, three combination functions (min, max, and mean) perform quite similarly. However, in all three experiments, the **mean** operator slightly exceeds the others in enhancing ensemble performance. The mean aggregation is also considered relatively stable because of its ability to reduce variance, leading to improved ensemble accuracy [1]. Therefore, we decide to use the **mean** combination function for all ensembles in the following experiments.

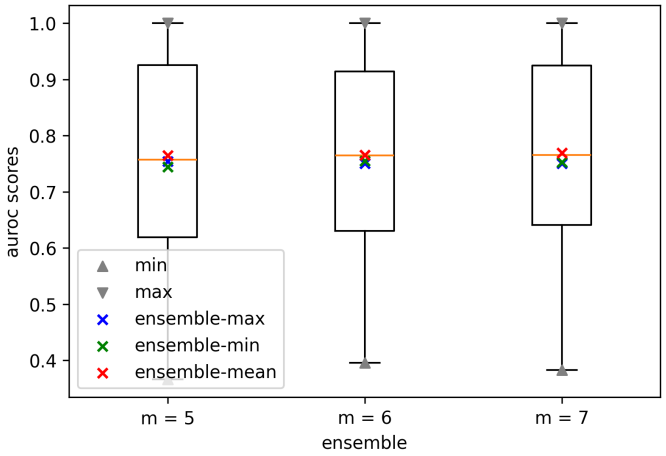


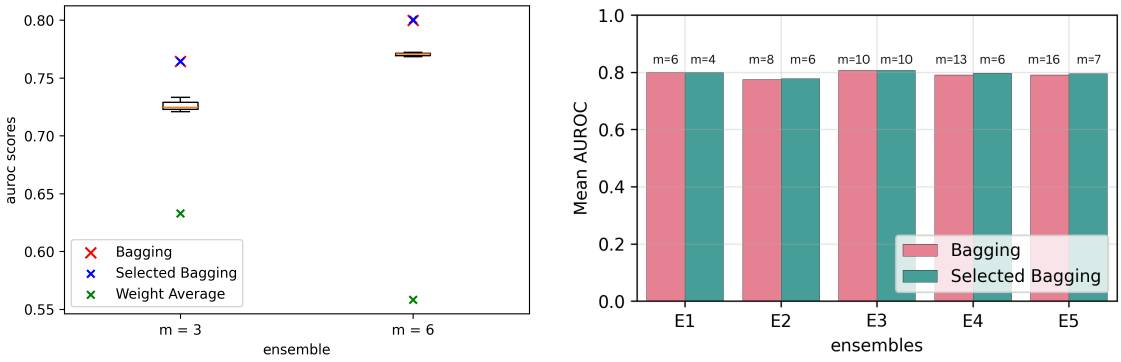
Figure 4.8: Performance of ensembles with 5, 6, and 7 base models on the ADBench benchmark. Boxplots show the distribution of AUROC scores, while Markers denote ensemble performance using different combination functions (min, max, and mean).

4.2.2.2 Which Ensemble Strategies outperform?

To evaluate the performance of three different ensemble strategies, we use two ensembles containing three and six base models, respectively. In the first ensemble, base models differ solely in the data prior type, whereas the components in the second ensemble differ only in the maximum number of clusters in a dataset. We choose this configuration for the experiment because the preconditions of the weight averaging strategy are relatively strict, requiring base models with the same architecture. The results in Figure 4.9(a) show that the **Weight Average** approach performs significantly worse than the other strategies.

Notably, its performance is even worse than that of any base model in the ensemble, indicating the inefficiency of the strategy and aligning with the findings of [66]. Therefore, even though the Weight Average strategy can help address memory and computational constraints, we decide not to study this strategy further in this work.

In contrast, both **Bagging** and **Selected Bagging** strategies demonstrate comparable performance and significantly outperform any individual base model in an ensemble. We further investigate these two approaches in a second experiment, as illustrated in Figure 4.9(b), where ensembles are formed by randomly sampling base models from the trained pool, and the ensemble size is varied. The results indicate that both strategies maintain similar performance levels across different configurations. Although the **Selected Bagging** approach mostly achieves slightly better results while requiring fewer base models (e.g., in ensembles E1, E2, E4, and E5), these gains are marginal. Moreover, Selected Bagging incurs significantly higher computational costs, as the number of candidate subsets grows combinatorially with the ensemble size. For instance, training an ensemble of 20 sub-models using Selected Bagging remained incomplete after few days, leading us to terminate the process. Despite this limitation during training, Selected Bagging offers an advantage at inference time, as the resulting ensembles typically contain fewer base models, leading to reduced inference latency compared to the Bagging strategy. Due to infrastructural resource constraints, we do not extend our investigation of Selected Bagging to larger ensembles. Nevertheless, the approach shows promise and might be worthy of further investigation when infrastructure is no longer a limitation. Overall, the results suggest that carefully selecting base models can substantially enhance the generalization capability of foundation model ensembles.



(a) Comparison of three strategies using ensembles with 3 and 6 base models.

(b) Comparison between Bagging and Selected Bagging strategies.

Figure 4.9: Performance of ensemble strategies on the ADBench benchmark. In Figure (a), Boxplots show the distribution of AUROC scores, whereas Markers denote the AUROC scores of ensembles deploying different strategies. Figure (b) illustrates the mean AUROC scores across five ensembles using Bagging and Selected Bagging strategies.

4.2.2.3 Do large ensembles perform better?

To assess the influence of ensemble size on performance, we train ensembles using the Bagging strategy. We start with an ensemble containing the top five best base models, and then add more base models randomly from the trained pool to the subsequent ensembles. Figure 4.10 demonstrates the mean AUROC scores of ensembles with various sizes. Even though increasing the ensemble size may lead to a reduction in performance at some point, ensembles generally still benefit from an increasing number of base models. The fluctuation in performance when increasing the ensemble size indicates that the selection of base models included in an ensemble also plays an important role in enhancing its generalization capacity.

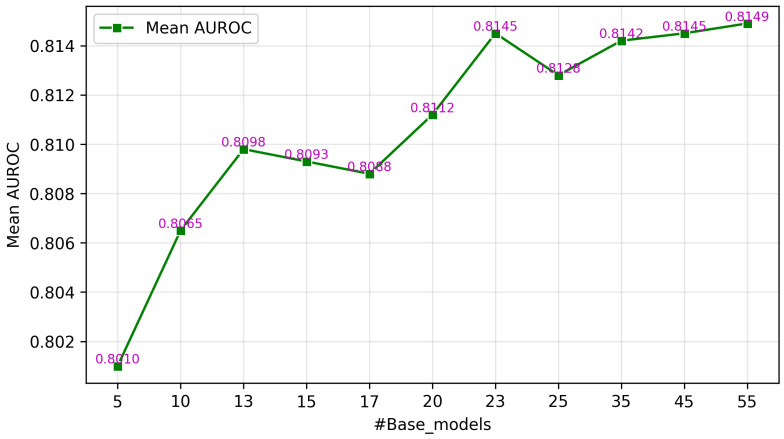


Figure 4.10: The impact of ensemble size on performance.

4.2.2.4 How does the diversity of base models affect on ensemble performance?

In this experiment, we construct five ensembles using the Bagging strategy, as illustrated in Figure 4.11. The first three ensembles each consist of five base models, while the remaining two contain ten models. The *Feat_80* and *Feat_70* ensembles are composed of models sharing the same CMM-based data prior and maximum feature dimensions of 80 and 70, respectively, with only minor variations in other hyperparameters. In contrast, sub-models in the *Diversity_5* ensemble are highly heterogeneous, with the maximum number of features ranging from 20 to 90, as well as different data priors and other hyperparameter settings. Similarly, the *Top10* ensemble contains the ten best-performing models with relatively similar characteristics, whereas *Diversity_10* is designed to maximize diversity among its sub-models. The degree of similarity or diversity within each ensemble is reflected in the spread of the corresponding boxplots.

Notably, despite including some underperforming sub-models, both the *Diversity_ 5* and *Diversity_ 10* ensembles outperform their respective homogeneous counterparts. This observation highlights the importance of diversity among base models. Individual ensemble members do not need to be very accurate but should differ sufficiently from each other to make different types of errors. Thereby, by combining the predictions of such a diverse set of models, the overall error of the ensemble can be reduced, enhancing its generalization capacity.

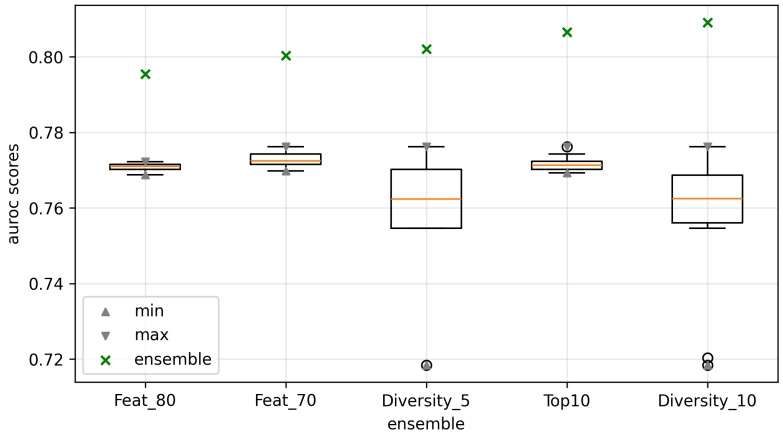


Figure 4.11: The impact of the diversity of base models on ensemble performance. Boxplots show the distribution of AUROC scores of base models included in an ensemble, while green Markers illustrate the AUROC scores of ensembles.

4.2.2.5 Comparison between our top ensembles and baseline methods

Here, we perform a comparison between our top ensembles using the Bagging and Selected Bagging strategies and 27 baseline methods. While we evaluate the published FoMo-0D model again on 57 ADBench datasets, the results of the remaining baselines in the semi-supervised setting are taken from [53].

Figure 4.12 demonstrates a comparison between our best ensembles and the 27 baselines in terms of performance and inference time. The 1st-, 2nd-, and 3rd-Bagging ensembles contain 55, 45, and 23 base models, respectively, whereas the 1st-Selected Bagging ensemble consists of 12 models selected from a candidate pool of 15. The results show that our approaches significantly outperform the FoMo-0D model and achieve performance that is comparable to or exceeds that of the top-4 baselines. Notably, the **1st-Bagging** ensemble achieves the best overall performance among all evaluated methods. In addition, all proposed ensembles demonstrate substantial reductions in inference time, underscoring their suitability for real-world anomaly detection scenarios. Although the **1st-Selected Bagging** ensemble attains slightly lower AUROC than the other ensembles, it offers the fastest

inference, highlighting an advantageous trade-off between performance and computational efficiency.

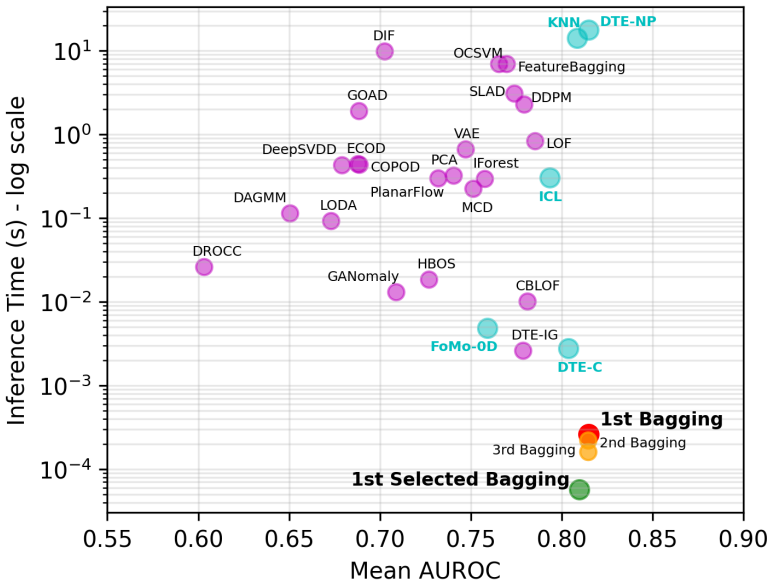


Figure 4.12: Average inference time vs. mean AUROC scores of our best ensembles compared to 27 baselines for all 57 ADBench datasets in the semi-supervised setting. Colour scheme: *red* (our 1st-Bagging ensemble), *orange* (our top two and three Bagging ensembles), *green* (our 1st-Selected Bagging ensemble), *blue* (FoMo-0D and the top-4 baselines), *pink* (the remaining baselines).

4.2.2.6 The two-sided Wilcoxon signed rank test

In the previous experiment, our **1st-Bagging** ensemble demonstrates performance that surpasses all baseline methods. To assess whether these improvements are statistically significant relative to FoMo-0D and the top-4 best-performing baselines, we conduct Wilcoxon signed rank tests [73]. In particular, we use a two-sided Wilcoxon signed rank test between the 1st-Bagging ensemble and each baseline using AUROC scores across all 57 ADBench datasets. The null hypothesis assumes that there is no significant difference between the paired performances, while the alternative hypothesis assumes a significant difference. The null hypothesis is rejected if the p-value falls below a multiple level of significance α [40], which we set at $\alpha = 0.05$ following convention.

Figure 4.13 illustrates p-values of the two-sided Wilcoxon signed rank test, in which **p-value** is transformed to $-\mathbf{log_{10}(p\text{-}value)}$ for visualization purposes. Details of all p-values are provided in Table A.1. Overall, the tests indicate that differences in the performances between our 1st-Bagging ensemble and ICL, DTE-C, and FoMo-0D baselines are **statistically significant** at $p < 0.05$ across all 57 ADBench datasets. However, the 1st-Bagging ensemble shows **no statistically significant difference** compared to DTE-NP and kNN baselines. Although there is no statistical evidence that our 1st-Bagging significantly out-

performs the top-2 DTE-NP and kNN baselines, its efficiency in inference time remains a remarkable improvement.

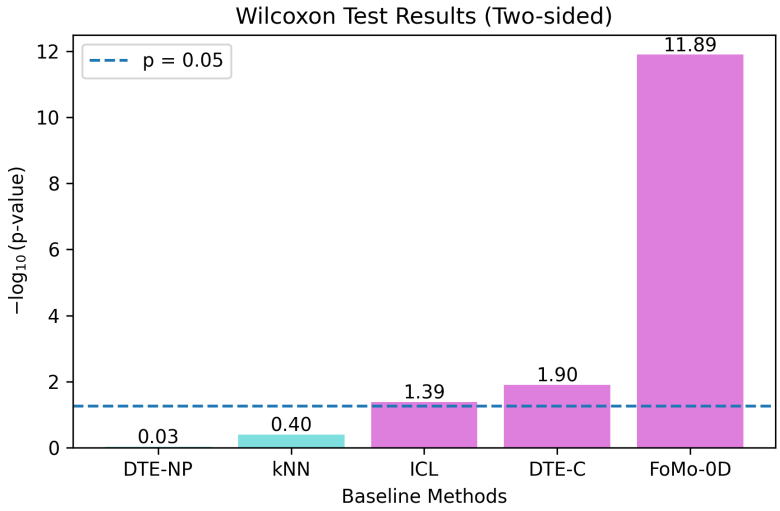


Figure 4.13: p-values of the two-sided Wilcoxon signed rank test, comparing our **1st-Bagging** ensemble to the top-4 baseline methods and FoMo-0D using AUROC scores across all 57 ADBench datasets. The p-values are transformed to $-\mathbf{log_{10}(p\text{-}value)}$ for visualization purposes. Values above the blue threshold line indicate statistically significant differences.

Chapter 5

Conclusion and future work

This thesis investigates ensemble methods for improving the generalization capability of foundation models for outlier detection in tabular data. Compared to conventional machine learning approaches, foundation models require substantial computational resources during both pre-training and inference phases, leading to challenges in constructing ensembles. These challenges are further exacerbated by hardware limitations, as our experiments are conducted on a NVIDIA RTX 4500 Ada Generation GPU with 24GB of GDDR6 memory, making the training of large-scale models infeasible.

To overcome these constraints, we develop computationally efficient FoMo-0D-based foundation models that achieve performance comparable to, or exceeding, that of the original FoMo-0D. We have shown that our base models can significantly outperform the original FoMo-0D while achieving substantial improvements in inference time at a small computational cost. This efficiency facilitates the construction of robust ensembles with low latency, making them particularly well-suited for real-time anomaly detection applications.

Notably, our **1st-Bagging** ensemble surpasses all 27 baseline methods across 57 AD-Bench datasets while maintaining substantially lower inference time. In addition, the **1st-Selected Bagging** ensemble achieves competitive performance, surpassing most baselines except DTE-NP and kNN, and provides the greatest gains in inference efficiency. Importantly, as our ensembles are composed of FoMo-0D-based foundation models, they retain the capability for **zero-shot** inference on unseen datasets without requiring further fine-tuning or retraining. This feature further highlights the suitability of our ensembles for real-world anomaly detection scenarios, where anomalies are rare, evolving, and often not predefined.

Due to infrastructure limitations, we cannot conduct a comparison between the Bagging and Selected Bagging strategies at a large scale. However, the results obtained on a small scale suggest that carefully selecting base models can substantially boost the generalization capability of foundation model ensembles while reducing inference latency. This indicates that the Selected Bagging strategy could be potential and valuable for further investiga-

tion when infrastructure is not a limitation. Additionally, the experiments suggest that increasing the number of base models can generally enhance ensemble performance, but diversity among the components is also crucial. By aggregating predictions from a diverse set of models, the ensemble can effectively reduce overall error and achieve improved generalization capacity.

In general, we have investigated three different ensemble strategies: Bagging, Selected Bagging, and Weight Average. While the Weight Averaging approach did not yield satisfactory results, both Bagging and Selected Bagging demonstrate substantial improvements in terms of performance and inference efficiency. Notably, our findings indicate that carefully optimizing and streamlining foundation models can effectively reduce computational requirements while enhancing generalization capability. This balance between efficiency and performance underscores the practical applicability of the proposed methods, aligning with the broader objective of developing deployable machine learning solutions.

For the future work, we propose training foundation models on a set of synthetic datasets generated from multiple data priors, rather than relying on a single prior. Such an approach could expose models to more diverse data distributions, thereby improving their generalization ability to new unseen datasets. Furthermore, our experimental results indicate that the selection of base models within an ensemble significantly affects overall performance, suggesting that individual models do not contribute equally. Therefore, we propose exploring weighted ensemble methods, where each base model is assigned a contribution weight based on its relative importance. By prioritizing more informative models, the approach has the potential to further enhance the robustness and generalization capability of ensembles.

Another promising direction for future work concerns the **Weight Average** ensemble strategy. Our experiments indicate that its performance is highly sensitive to the selection of base models. Due to computational limitations, we were unable to evaluate this strategy using a sufficiently large candidate pool. Hence, we propose investigating it further in future work. If its performance can be further optimized, the Weight Average strategy could become an attractive solution for real-world applications that demand highly efficient models with real-time inference capabilities.

List of Figures

2.1 An example of a heterogeneous tabular data, whose variables are selected from the adult dataset [19], [9] 7

2.2 Transformer model architecture [70] 9

2.3 The outline of Prior-Data Fitted Networks [54] 11

2.4 Comparison of typical OD vs. the FoMo-0D settings [63] 13

2.5 The router mechanism for scalable attention employed in FoMo-0D architecture [63] 14

2.6 Illustration of the bias-variance trade-off and model optimization [27]. This relationship is similar to the relationship between overfitting, underfitting, and model capacity. Increasing model capacity may lead to a decrease in bias, but an increase in variance, resulting in a different curve for generalization error. Therefore, finding an optimal capacity is important. 18

2.7 The ROC curves of three different models [72]. The diagonal line represents a random guessing model. The closer a model's ROC curve is to the diagonal line, the less accurate it is. Accordingly, the model corresponding to the blue curve performs best, whereas the one corresponding to the orange curve performs worst. 19

2.8 The difference between comparing algorithms in ROC and PR curves [16] . 20

3.1 Demonstration of synthetic data in 2D drawn from a GMM with 80th percentile as the criterion [63] 23

3.2 Illustration of two-dimensional synthetic datasets drawn from a MGMM with the 90th percentile as the criterion and CMM. 24

3.3 Overview of the proposed framework for constructing ensembles of foundation models. In the first stage, multiple base models are trained using the FoMo-0D architecture. In the second stage, these models are combined through a unified decision-making mechanism, where three different ensemble strategies are explored. 27

4.1 The influence of data priors on base models in terms of performance and training time. Figure (a) demonstrates the mean AUROC scores of three model groups categorized by their data priors. Figure (b) presents the model training time (in seconds) of each group. 35

4.2 The behavior of the learning rate. Figure (a) illustrates the relationship between learning rate and mean AUROC. Figure (b) shows the training loss curves of the models, highlighting their convergence behavior under different learning rates. 36

4.3 The influence of the number of epochs on base models. Figure (a) shows how mean AUROC and training time shift when increasing epochs. Figure (b) displays a comparison of training losses between the base models with 50 and 100 epochs, respectively. It also provides the learning rates at the end of the pre-training phase for both models. Figure (c) and (d) illustrate Training and validation losses of the model with 50 and 100 epochs, respectively. . . 37

4.4 The impact of batch sizes on model performance and training time. Figure (a) illustrates mean AUROC scores for varying values of max feature dimension and batch size. Figure (b) highlights that increasing the batch size leads to an increase in the model training time. 39

4.5 Illustrate the impact of nlayer and nhead hyperparameters. Figure (a) indicates the effect of increasing the number of layers in the transformer encoder block on model performance. Figure (b) demonstrates that two models with nlayer of 6 and 7 have nearly identical training loss curves and validation loss curves. Figure (c) shows that model performance increases when the number of attention heads is reduced. 40

4.6 Average inference time vs. mean AUROC scores of the original FoMo-0D model (blue) and our best base model (orange) for all 57 ADBench dtatasets. Additional points correspond to other base models ranked among the top 10 in performance. 42

4.7 Mean AUROC comparison between the FoMo-0D and our best base model across ten complex datasets from CV and NLP domains in the ADBench benchmark. For each dataset, results are reported per class, with each point representing the mean AUROC of a single class and boxplots summarizing the distribution across all classes. 42

4.8 Performance of ensembles with 5, 6, and 7 base models on the ADBench benchmark. Boxplots show the distribution of AUROC scores, while Markers denote ensemble performance using different combination functions (min, max, and mean). 43

4.9 Performance of ensemble strategies on the ADBench benchmark. In Figure (a), Boxplots show the distribution of AUROC scores, whereas Markers denote the AUROC scores of ensembles deploying different strategies. Figure (b) illustrates the mean AUROC scores across five ensembles using Bagging and Selected Bagging strategies. 44
4.10 The impact of ensemble size on performance. 45
4.11 The impact of the diversity of base models on ensemble performance. Boxplots show the distribution of AUROC scores of base models included in an ensemble, while green Markers illustrate the AUROC scores of ensembles. . 46
4.12 Average inference time vs. mean AUROC scores of our best ensembles compared to 27 baselines for all 57 ADBench datasets in the semi-supervised setting. Colour scheme: *red* (our 1st-Bagging ensemble), *orange* (our top two and three Bagging ensembles), *green* (our 1st-Selected Bagging ensemble), *blue* (FoMo-0D and the top-4 baselines), *pink* (the remaining baselines). 47
4.13 p-values of the two-sided Wilcoxon signed rank test, comparing our **1st-Bagging** ensemble to the top-4 baseline methods and FoMo-0D using AUROC scores across all 57 ADBench datasets. The p-values are transformed to $-\mathbf{log_{10}(p\text{-}value)}$ for visualization purposes. Values above the blue threshold line indicate statistically significant differences. 48

List of Tables

4.1 Description of all tabular outlier detection datasets in ADBench [32]. The last ten datasets in the table are image and text datasets that are vectorized through pre-trained ResNet18 [36] and BERT [45] models, respectively. . . . 32
4.2 The hyperparameter settings of the original FoMo-0D model and our best base model 41

A.1 p-values of the two-sided Wilcoxon signed rank test, comparing our **1st-Bagging** ensemble to the top-4 baseline methods and FoMo-0D using AUROC scores across all 57 ADBench datasets. 64
A.2 Average AUROC scores of the FoMo-0D, our top-1 base model, top-3 Bagging ensembles and top-1 Selected Bagging ensemble across all ADBench datasets. 65
A.3 Average AUCPR scores of the FoMo-0D, our top-1 base model, top-3 Bagging ensembles and top-1 Selected Bagging ensemble across all ADBench datasets. 67
A.4 Average F1 scores of the FoMo-0D, our top-1 base model, top-3 Bagging ensembles and top-1 Selected Bagging ensemble across all ADBench datasets. 69

Bibliography

[1] Aggarwal, Charu C: *An introduction to outlier analysis.* In *Outlier analysis*, pages 1–34. Springer, 2016.

[2] Aggarwal, Charu C and Philip S Yu: *Outlier detection for high dimensional data.* In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data*, pages 37–46, 2001.

[3] Akcay, Samet, Amir Atapour-Abarghouei and Toby P Breckon: *Ganomaly: Semi-supervised anomaly detection via adversarial training.* In *Asian conference on computer vision*, pages 622–637. Springer, 2018.

[4] Akyürek, Ekin, Dale Schuurmans, Jacob Andreas, Tengyu Ma and Denny Zhou: *What learning algorithm is in-context learning? investigations with linear models.* arXiv preprint arXiv:2211.15661, 2022.

[5] Arik, Sercan Ö and Tomas Pfister: *Tabnet: Attentive interpretable tabular learning.* In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 6679–6687, 2021.

[6] Barnett, Vic, Toby Lewis et al.: *Outliers in statistical data*, volume 3. Wiley New York, 1994.

[7] Bergman, Liron and Yedid Hoshen: *Classification-based anomaly detection for general data.* arXiv preprint arXiv:2005.02359, 2020.

[8] Bommasani, Rishi: *On the opportunities and risks of foundation models.* arXiv preprint arXiv:2108.07258, 2021.

[9] Borisov, Vadim, Tobias Leemann, Kathrin Sessler, Johannes Haug, Martin Pawelczyk and Gjergji Kasneci: *Deep neural networks and tabular data: A survey.* IEEE transactions on neural networks and learning systems, 35(6):7499–7519, 2022.

[10] Breiman, Leo: *Bagging predictors.* Machine learning, 24:123–140, 1996.

[11] Breunig, Markus M, Hans-Peter Kriegel, Raymond T Ng and Jörg Sander: *LOF: identifying density-based local outliers.* In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 93–104, 2000.

[12] Buczak, Anna L and Erhan Guven: *A survey of data mining and machine learning methods for cyber security intrusion detection.* IEEE Communications surveys & tutorials, 18(2):1153–1176, 2015.

[13] Cartella, Francesco, Orlando Anunciacao, Yuki Funabiki, Daisuke Yamaguchi, Toru Akishita and Olivier Elshocht: *Adversarial attacks for tabular data: Application to fraud detection and imbalanced data.* arXiv preprint arXiv:2101.08030, 2021.

[14] Chicco, Davide and Giuseppe Jurman: *The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation.* BMC genomics, 21(1):6, 2020.

[15] Chui, Michael, James Manyika, Mehdi Miremadi, Nicolaus Henke, Rita Chung, Pieter Nel and Sankalp Malhotra: *Notes from the AI frontier: Insights from hundreds of use cases.* McKinsey Global Institute, 2(267):1–31, 2018.

[16] Davis, Jesse and Mark Goadrich: *The relationship between Precision-Recall and ROC curves.* In *Proceedings of the 23rd international conference on Machine learning*, pages 233–240, 2006.

[17] Dietterich, Thomas G: *Ensemble methods in machine learning.* In *International workshop on multiple classifier systems*, pages 1–15. Springer, 2000.

[18] Ding, Xueying, Lingxiao Zhao and Leman Akoglu: *Hyperparameter sensitivity in deep outlier detection: Analysis and a scalable hyper-ensemble solution.* Advances in Neural Information Processing Systems, 35:9603–9616, 2022.

[19] Dua, Dheeru, Casey Graff et al.: *UCI machine learning repository.* 2017.

[20] Fauconnier, Cecile and Gentiane Haesbroeck: *Outliers detection with the minimum covariance determinant estimator in practice.* Statistical Methodology, 6(4):363–379, 2009.

[21] Fitkov-Norris, Elena, Samireh Vahid and Chris Hand: *Evaluating the impact of categorical data encoding and scaling on neural network classification performance: The case of repeat consumption of identical cultural goods.* In *International Conference on Engineering Applications of Neural Networks*, pages 343–352. Springer, 2012.

[22] Freund, Yoav and Robert E Schapire: *A desicion-theoretic generalization of on-line learning and an application to boosting.* In *European conference on computational learning theory*, pages 23–37. Springer, 1995.

[23] Gao, Jing and Pang-Ning Tan: *Converting output scores from outlier detection algorithms into probability estimates.* In *Sixth International Conference on Data Mining (ICDM'06)*, pages 212–221. IEEE, 2006.

[24] Garces, Hugo and Daniel Sbarbaro: *Outliers detection in environmental monitoring databases.* Engineering Applications of Artificial Intelligence, 24(2):341–349, 2011.

[25] Goadrich, Mark, Louis Oliphant and Jude Shavlik: *Gleaner: Creating ensembles of first-order clauses to improve recall-precision curves.* Machine Learning, 64(1):231–261, 2006.

[26] Goldstein, Markus and Andreas Dengel: *Histogram-based outlier score (hbos): A fast unsupervised anomaly detection algorithm.* KI-2012: poster and demo track, 1:59–63, 2012.

[27] Goodfellow, Ian, Yoshua Bengio and Aaron Courville: *Deep Learning.* MIT Press, 2016. `http://www.deeplearningbook.org`.

[28] Gorishniy, Yury, Ivan Rubachev and Artem Babenko: *On embeddings for numerical features in tabular deep learning.* Advances in Neural Information Processing Systems, 35:24991–25004, 2022.

[29] Goyal, Sachin, Aditi Raghunathan, Moksh Jain, Harsha Vardhan Simhadri and Prateek Jain: *DROCC: Deep robust one-class classification.* In *International conference on machine learning*, pages 3711–3721. PMLR, 2020.

[30] Graves, Alex: *Generating sequences with recurrent neural networks.* arXiv preprint arXiv:1308.0850, 2013.

[31] Han, Jiawei, Jian Pei and Hanghang Tong: *Data mining: concepts and techniques.* Morgan kaufmann, 2022.

[32] Han, Songqiao, Xiyang Hu, Hailiang Huang, Minqi Jiang and Yue Zhao: *Adbench: Anomaly detection benchmark.* Advances in Neural Information Processing Systems, 35:32142–32159, 2022.

[33] Han, Zeyu, Chao Gao, Jinyang Liu, Jeff Zhang and Sai Qian Zhang: *Parameter-efficient fine-tuning for large models: A comprehensive survey.* arXiv preprint arXiv:2403.14608, 2024.

[34] Hansen, Lars Kai and Peter Salamon: *Neural network ensembles.* IEEE transactions on pattern analysis and machine intelligence, 12(10):993–1001, 1990.

[35] Hastie, Trevor, Robert Tibshirani, Jerome H Friedman and Jerome H Friedman: *The elements of statistical learning: data mining, inference, and prediction*, volume 2. Springer, 2009.

[36] He, Kaiming, Xiangyu Zhang, Shaoqing Ren and Jian Sun: *Deep residual learning for image recognition.* In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[37] He, Zengyou, Xiaofei Xu and Shengchun Deng: *Discovering cluster-based local outliers.* Pattern recognition letters, 24(9-10):1641–1650, 2003.

[38] Ho, Jonathan, Ajay Jain and Pieter Abbeel: *Denoising diffusion probabilistic models.* Advances in neural information processing systems, 33:6840–6851, 2020.

[39] Hollmann, Noah, Samuel Müller, Katharina Eggensperger and Frank Hutter: *Tabpfn: A transformer that solves small tabular classification problems in a second.* arXiv preprint arXiv:2207.01848, 2022.

[40] Holm, Sture: *A simple sequentially rejective multiple test procedure.* Scandinavian journal of statistics, pages 65–70, 1979.

[41] Houlsby, Neil, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan and Sylvain Gelly: *Parameter-efficient transfer learning for NLP.* In *International conference on machine learning*, pages 2790–2799. PMLR, 2019.

[42] Huyen, Chip: *AI engineering: Building applications with foundation models.* " O'Reilly Media, Inc.", 2024.

[43] Izmailov, Pavel, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov and Andrew Gordon Wilson: *Averaging weights leads to wider optima and better generalization.* arXiv preprint arXiv:1803.05407, 2018.

[44] Jospin, Laurent Valentin, Hamid Laga, Farid Boussaid, Wray Buntine and Mohammed Bennamoun: *Hands-on Bayesian neural networks—A tutorial for deep learning users.* IEEE Computational Intelligence Magazine, 17(2):29–48, 2022.

[45] Kenton, Jacob Devlin Ming-Wei Chang, Lee Kristina Toutanova et al.: *Bert: Pre-training of deep bidirectional transformers for language understanding.* In *Proceedings of naacL-HLT*, volume 1, pages 4171–4186. Minneapolis, Minnesota, 2019.

[46] Keskar, Nitish Shirish, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy and Ping Tak Peter Tang: *On large-batch training for deep learning: Generalization gap and sharp minima.* arXiv preprint arXiv:1609.04836, 2016.

[47] Kingma, Diederik P and Max Welling: *Auto-encoding variational bayes.* arXiv preprint arXiv:1312.6114, 2013.

[48] Kyuseok, Shim: *Efficient algorithms for mining outliers from large data sets.* In *ACM SIGMOD International Conference on Management of Data*, 2000.

[49] Lazarevic, Aleksandar and Vipin Kumar: *Feature bagging for outlier detection.* In *Proceedings of the eleventh ACM SIGKDD international conference on Knowledge discovery in data mining*, pages 157–166, 2005.

[50] Li, Zheng, Yue Zhao, Nicola Botta, Cezar Ionescu and Xiyang Hu: *Copod: copula-based outlier detection.* In *2020 IEEE international conference on data mining (ICDM)*, pages 1118–1123. IEEE, 2020.

[51] Li, Zheng, Yue Zhao, Xiyang Hu, Nicola Botta, Cezar Ionescu and George H Chen: *Ecod: Unsupervised outlier detection using empirical cumulative distribution functions.* IEEE Transactions on Knowledge and Data Engineering, 35(12):12181–12193, 2022.

[52] Liu, Fei Tony, Kai Ming Ting and Zhi-Hua Zhou: *Isolation forest.* In *2008 eighth ieee international conference on data mining*, pages 413–422. IEEE, 2008.

[53] Livernoche, Victor, Vineet Jain, Yashar Hezaveh and Siamak Ravanbakhsh: *On diffusion modeling for anomaly detection.* arXiv preprint arXiv:2305.18593, 2023.

[54] Müller, Samuel, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka and Frank Hutter: *Transformers can do bayesian inference.* arXiv preprint arXiv:2112.10510, 2021.

[55] Pang, Guansong, Chunhua Shen, Longbing Cao and Anton Van Den Hengel: *Deep learning for anomaly detection: A review.* ACM computing surveys (CSUR), 54(2):1–38, 2021.

[56] Patro, SGOPAL and Kishore Kumar Sahu: *Normalization: A preprocessing stage.* arXiv preprint arXiv:1503.06462, 2015.

[57] Pevnỳ, Tomáš: *Loda: Lightweight on-line detector of anomalies.* Machine Learning, 102(2):275–304, 2016.

[58] Powers, David MW: *Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation.* arXiv preprint arXiv:2010.16061, 2020.

[59] Rezende, Danilo and Shakir Mohamed: *Variational inference with normalizing flows.* In *International conference on machine learning*, pages 1530–1538. PMLR, 2015.

[60] Rose, Karen, Scott Eldridge, Lyman Chapin et al.: *The internet of things: An overview.* The internet society (ISOC), 80(15):1–53, 2015.

[61] Ruff, Lukas, Robert Vandermeulen, Nico Goernitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller and Marius Kloft: *Deep one-class classification.* In *International conference on machine learning*, pages 4393–4402. PMLR, 2018.

[62] Schölkopf, Bernhard, Robert C Williamson, Alex Smola, John Shawe-Taylor and John Platt: *Support vector method for novelty detection.* Advances in neural information processing systems, 12, 1999.

[63] Shen, Yuchen, Haomin Wen and Leman Akoglu: *FoMo-0D: A Foundation Model for Zero-shot Outlier Detection.* In *1st ICML Workshop on Foundation Models for Structured Data.*

[64] Shenkar, Tom and Lior Wolf: *Anomaly detection for tabular data with internal contrastive learning.* In *International conference on learning representations*, 2022.

[65] Shyu, Mei-Ling, Shu-Ching Chen, Kanoksri Sarinnapakorn and LiWu Chang: *A novel anomaly detection scheme based on principal component classifier.* 2003.

[66] Singh, Sidak Pal and Martin Jaggi: *Model fusion via optimal transport.* Advances in Neural Information Processing Systems, 33:22045–22055, 2020.

[67] Sutskever, Ilya, Oriol Vinyals and Quoc V Le: *Sequence to sequence learning with neural networks.* Advances in neural information processing systems, 27, 2014.

[68] Thrun, Sebastian: *Lifelong learning algorithms.* In *Learning to learn*, pages 181–209. Springer, 1998.

[69] Van Capelleveen, Guido, Mannes Poel, Roland M Mueller, Dallas Thornton and Jos van Hillegersberg: *Outlier detection in healthcare fraud: A case study in the Medicaid dental domain.* International journal of accounting information systems, 21:18–31, 2016.

[70] Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser and Illia Polosukhin: *Attention is all you need.* Advances in neural information processing systems, 30, 2017.

[71] Wang, Lidong: *Heterogeneous data and big data analytics.* In *ACIS*, volume 3, pages 8–15, 2017.

[72] Wikipedia: *Receiver operating characteristic.* `https://en.wikipedia.org/wiki/Receiver_operating_characteristic`. Accessed: 2023-08-16.

[73] Wilcoxon, Frank: *Individual comparisons by ranking methods.* Biometrics bulletin, 1(6):80–83, 1945.

[74] Wolpert, David H: *Stacked generalization.* Neural networks, 5(2):241–259, 1992.

[75] Wortsman, Mitchell, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith et al.: *Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time.* In *International conference on machine learning*, pages 23965–23998. PMLR, 2022.

[76] Xin, Yi, Jianjiang Yang, Siqi Luo, Yuntao Du, Qi Qin, Kangrui Cen, Yangfan He, Zhiwei Zhang, Bin Fu, Xiaokang Yang et al.: *Parameter-efficient fine-tuning for pre-trained vision models: A survey and benchmark.* arXiv preprint arXiv:2402.02242, 2024.

[77] Xu, Hongzuo, Guansong Pang, Yijie Wang and Yongjun Wang: *Deep isolation forest for anomaly detection.* IEEE transactions on knowledge and data engineering, 35(12):12591–12604, 2023.

[78] Xu, Hongzuo, Yijie Wang, Juhui Wei, Songlei Jian, Yizhou Li and Ning Liu: *Fascinating supervisory signals and where to find them: Deep anomaly detection with scale learning.* In *International Conference on Machine Learning*, pages 38655–38673. PMLR, 2023.

[79] Zhang, Dan, Tao Feng, Lilong Xue, Yuandong Wang, Yuxiao Dong and Jie Tang: *Parameter-efficient fine-tuning for foundation models.* arXiv preprint arXiv:2501.13787, 2025.

[80] Zhang, Yunhao and Junchi Yan: *Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting.* In *The eleventh international conference on learning representations*, 2023.

[81] Zhou, Xiongtao, Jie He, Yuhua Ke, Guangyao Zhu, Víctor Gutiérrez-Basulto and Jeff Pan: *An empirical study on parameter-efficient fine-tuning for multimodal large language models.* In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10057–10084, 2024.

[82] Zimek, Arthur, Erich Schubert and Hans-Peter Kriegel: *A survey on unsupervised outlier detection in high-dimensional numerical data.* Statistical Analysis and Data Mining: The ASA Data Science Journal, 5(5):363–387, 2012.

[83] Zong, Bo, Qi Song, Martin Renqiang Min, Wei Cheng, Cristian Lumezanu, Daeki Cho and Haifeng Chen: *Deep autoencoding gaussian mixture model for unsupervised anomaly detection.* In *International conference on learning representations*, 2018.

Appendix

Appendix A

A.1 p-values

Table A.1: p-values of the two-sided Wilcoxon signed rank test, comparing our **1st-Bagging** ensemble to the top-4 baseline methods and FoMo-0D using AUROC scores across all 57 ADBench datasets.

Test	**p-value**
DTE-NP	0.939
kNN	0.401
ICL	**0.043**
DTE-C	**0.012**
FoMo-0D	$\mathbf{1.281 \times 10^{-12}}$

A.2 Average AUROC, AUCPR, and F1 Scores

Tables A.2, A.3, and A.4 present detailed results for average AUROC, AUCPR, and F1 scores, respectively. These include the re-evaluated performance of the FoMo-0D model on 57 ADBench datasets, as well as results from our top-1 base model, top-3 Bagging ensembles, and top-1 Selected Bagging ensemble. As datasets such as CIFAR10, Fashion-MNIST, MNIST-C, MVTec-AD, SVHN, Agnews, and 20news are partitioned into multiple categories, we report scores at the category level to ensure consistency and mitigate data aggregation bias. Therefore, the table reports results for 121 dataset instances rather than the original 57 datasets.

A.2.1 Average AUROC Scores

Table A.2: Average AUROC scores of the FoMo-0D, our top-1 base model, top-3 Bagging ensembles and top-1 Selected Bagging ensemble across all ADBench datasets.

datasets	FoMo-0D	1st base model	1st Bagging	2nd Bagging	3rd Bagging	1st Selected Bagging
mammography	71.00	77.97	80.07	79.49	80.57	77.88
vowels	81.29	70.40	82.80	81.39	85.69	85.44
optdigits	85.28	91.83	94.43	93.67	91.74	92.84
thyroid	96.84	96.01	96.52	96.66	96.47	96.35
WBC	99.75	99.58	99.49	99.49	99.33	99.26
Stamps	97.11	98.42	98.61	98.64	98.69	98.44
Cardiotocography	72.15	60.01	64.05	64.01	63.32	63.08
fault	61.82	56.36	57.43	57.47	58.85	57.02
census	61.35	57.49	61.73	62.21	61.99	60.25
skin	71.39	93.83	96.38	96.46	95.53	96.16
InternetAds	61.51	76.18	80.01	80.34	79.62	80.40
breastw	99.40	93.29	98.91	98.80	98.78	98.73
Waveform	71.02	64.73	68.48	68.96	65.52	65.28
vertebral	90.22	94.30	95.18	95.16	94.22	93.95
landsat	69.53	61.06	57.86	56.85	60.99	63.21
musk	99.96	54.11	95.03	98.53	97.93	92.63
Ionosphere	95.37	93.74	97.54	96.66	97.62	97.41
Hepatitis	100.00	100.00	100.00	100.00	100.00	100.00
fraud	95.82	90.98	96.07	96.13	95.95	94.71
Wilt	97.13	94.35	95.25	95.30	94.40	94.36
WDBC	99.84	99.71	99.56	99.59	99.54	99.47
cardio	94.27	83.30	91.94	91.23	91.54	91.09
campaign	66.20	63.62	71.82	72.20	67.95	64.74
yeast	49.91	44.94	46.04	46.24	45.83	45.45
PageBlocks	86.58	82.99	87.58	87.96	84.73	84.64
backdoor	92.18	96.69	95.89	96.72	96.65	94.19
WPBC	77.18	92.80	94.47	94.25	92.89	92.68
smtp	79.22	79.16	80.15	80.00	80.87	78.95
ALOI	54.51	52.14	53.96	53.32	52.84	52.42
shuttle	99.73	99.59	99.91	99.90	99.91	99.87
mnist	91.27	91.12	92.97	92.40	92.62	92.45
speech	42.93	44.80	36.77	37.54	39.96	40.29
pendigits	98.11	97.45	98.89	98.82	98.69	98.51
magic.gamma	84.97	86.25	87.00	86.69	87.37	87.45
Lymphography	100.00	100.00	100.00	100.00	100.00	100.00
annthyroid	85.17	81.40	82.94	83.12	82.06	81.77
glass	97.99	99.50	99.76	99.71	99.69	99.71
satellite	84.98	80.08	76.76	76.56	80.81	81.53
http	99.95	98.84	99.36	99.22	98.99	99.58
cover	99.58	99.49	99.58	99.57	99.49	99.39
satimage-2	98.40	97.72	98.90	98.90	98.97	98.63
Pima	78.65	78.34	80.63	80.64	79.43	79.37
celeba	74.95	60.36	64.67	68.07	69.19	61.35
SpamBase	78.21	82.24	84.04	83.10	84.71	84.48
letter	40.90	63.72	48.41	47.66	49.07	52.70
donors	99.92	99.91	99.98	99.94	99.96	99.97
wine	100.00	100.00	100.00	100.00	100.00	100.00
CIFAR10_0	72.45	69.57	72.67	74.35	74.60	74.59
CIFAR10_1	62.20	60.79	68.18	68.60	66.07	67.18
CIFAR10_2	60.71	56.97	63.00	63.37	63.31	60.50
CIFAR10_3	54.68	50.25	57.26	57.86	57.32	53.12
CIFAR10_4	67.97	67.86	76.47	76.42	75.32	75.25
CIFAR10_5	55.25	48.86	56.44	56.71	56.99	51.10
CIFAR10_6	68.35	62.06	73.40	73.05	72.28	71.96
CIFAR10_7	61.23	60.69	67.52	69.53	65.99	65.05
CIFAR10_8	70.49	67.46	72.72	73.74	71.51	73.06
CIFAR10_9	70.14	71.04	74.28	74.58	71.92	73.79

Continued on next page

datasets	FoMo-0D	1st base model	1st Bagging	2nd Bagging	3rd Bagging	1st Selected Bagging
FashionMNIST_0	86.43	83.32	90.86	90.96	91.16	90.86
FashionMNIST_1	93.43	88.65	98.84	98.81	98.61	98.47
FashionMNIST_2	84.04	73.04	87.56	87.30	86.79	86.32
FashionMNIST_3	85.65	85.54	93.61	92.84	92.97	91.96
FashionMNIST_4	83.89	78.64	87.30	87.61	86.12	85.03
FashionMNIST_5	92.49	91.89	95.36	95.51	95.53	95.81
FashionMNIST_6	73.13	71.22	80.01	78.93	80.13	79.16
FashionMNIST_7	93.63	90.74	97.26	97.34	97.08	96.32
FashionMNIST_8	84.29	85.00	88.22	88.22	88.72	88.57
FashionMNIST_9	95.81	94.37	97.16	96.82	97.24	96.95
MNIST-C_rotate	54.73	59.87	61.92	63.06	61.86	62.21
MNIST-C_brightness	73.27	81.53	89.23	85.92	87.70	87.72
MNIST-C_scale	69.47	88.39	90.82	91.05	90.88	91.11
MNIST-C_spatter	88.10	88.90	93.23	92.90	92.41	93.05
MNIST-C_shear	65.85	68.24	72.40	73.65	71.27	70.09
MNIST-C_translate	63.93	71.34	76.24	78.17	76.44	74.98
MNIST-C_shot_noise	81.55	76.88	90.89	90.34	87.75	87.58
MNIST-C_zigzag	85.42	83.01	94.52	94.32	93.18	93.80
MNIST-C_identity	47.05	48.26	46.87	45.46	45.96	44.66
MNIST-C_dotted_line	79.60	76.45	91.61	91.23	89.58	90.96
MNIST-C_motion_blur	81.95	87.96	94.56	94.60	94.51	93.76
MNIST-C_glass_blur	94.66	90.22	99.42	99.21	99.11	98.84
MNIST-C_fog	93.74	94.69	98.92	98.71	99.01	99.25
MNIST-C_impulse_noise	99.99	99.50	100.00	100.00	100.00	99.98
MNIST-C_canny_edges	83.89	84.94	91.57	90.97	91.26	91.69
MNIST-C_stripe	99.95	98.85	99.98	99.98	99.98	99.96
MVTec-AD_pill	76.27	87.26	89.40	88.95	88.00	88.52
MVTec-AD_cable	71.62	93.95	93.92	93.78	94.82	93.66
MVTec-AD_capsule	61.30	82.70	89.60	89.95	92.16	89.57
MVTec-AD_leather	96.86	96.38	98.94	98.74	98.89	98.58
MVTec-AD_hazelnut	64.46	85.98	90.85	89.98	90.24	90.23
MVTec-AD_wood	78.24	90.78	95.25	95.33	95.79	93.56
MVTec-AD_carpet	71.84	87.20	87.24	87.63	88.07	87.39
MVTec-AD_metal_nut	78.06	85.25	94.08	93.09	92.48	90.99
MVTec-AD_transistor	66.63	90.90	96.40	97.00	95.55	96.11
MVTec-AD_zipper	89.11	94.49	96.80	96.83	96.77	97.21
MVTec-AD_grid	65.84	88.82	90.91	90.09	91.51	89.19
MVTec-AD_toothbrush	74.21	91.88	98.82	98.45	98.78	97.94
MVTec-AD_screw	52.10	73.84	84.25	84.61	84.89	84.62
MVTec-AD_bottle	97.14	99.62	99.35	99.15	98.89	98.87
MVTec-AD_tile	82.87	97.13	96.45	96.29	97.01	97.19
SVHN_0	57.59	53.64	63.71	65.85	66.22	64.62
SVHN_1	59.59	53.45	63.31	66.00	63.84	62.67
SVHN_2	64.01	63.70	65.81	64.69	66.13	64.69
SVHN_3	61.63	65.29	63.12	62.75	62.84	63.34
SVHN_4	58.74	61.83	62.25	62.80	64.19	63.34
SVHN_5	66.11	67.27	66.11	65.44	65.32	66.48
SVHN_6	54.08	55.99	55.88	56.82	56.34	58.65
SVHN_7	59.45	59.86	64.25	65.81	65.12	62.87
SVHN_8	56.35	57.82	60.09	60.57	60.48	59.70
SVHN_9	56.90	54.32	56.82	58.49	57.62	58.38
agnews_0	57.84	60.83	65.94	63.63	66.85	68.29
agnews_1	58.73	71.69	77.07	76.74	76.54	75.87
agnews_2	70.25	76.61	78.20	79.18	78.49	78.39
agnews_3	56.82	67.58	71.30	70.79	72.61	69.25
20news_0	64.34	74.65	79.60	77.46	78.19	76.94
20news_1	51.09	51.55	56.39	57.78	57.14	54.21
20news_2	51.45	50.16	52.90	51.83	51.34	50.67
20news_3	77.40	83.26	84.16	84.09	81.98	80.98
20news_4	58.23	56.07	58.29	56.09	60.80	61.81
20news_5	53.11	53.87	61.65	56.94	63.27	59.19
amazon	49.66	54.86	57.47	57.37	58.42	60.95
imdb	44.63	48.96	49.43	48.65	51.21	51.60
yelp	57.20	64.32	66.52	64.41	64.92	67.89

A.2.2 Average AUCPR scores

Table A.3: Average AUCPR scores of the FoMo-0D, our top-1 base model, top-3 Bagging ensembles and top-1 Selected Bagging ensemble across all ADBench datasets.

datasets	FoMo-0D	1st base model	1st Bagging	2nd Bagging	3rd Bagging	1st Selected Bagging
mammography	37.94	31.91	45.06	44.62	44.64	41.91
vowels	24.10	10.46	24.30	20.55	27.65	26.55
optdigits	31.88	28.05	40.16	38.39	34.64	33.12
thyroid	67.00	61.01	56.68	61.15	58.22	55.98
WBC	96.77	89.25	87.90	87.90	90.98	86.56
Stamps	92.13	85.53	83.92	85.44	85.06	83.08
Cardiotocography	63.55	44.64	53.25	53.09	54.09	49.69
fault	63.08	58.42	62.66	61.81	63.77	62.39
census	14.80	14.67	14.37	14.74	14.46	14.54
skin	49.51	82.07	91.92	92.13	88.86	89.99
InternetAds	40.93	61.48	73.27	71.78	72.56	70.58
breastw	99.34	87.34	96.04	95.97	95.69	95.91
Waveform	9.95	9.03	10.13	10.48	9.77	9.46
vertebral	69.74	70.95	74.24	73.80	73.89	72.98
landsat	58.24	55.92	54.55	54.67	55.99	56.98
musk	99.46	6.03	37.55	69.76	60.43	31.94
Ionosphere	96.95	87.95	96.63	95.09	97.61	94.81
Hepatitis	100.00	100.00	100.00	100.00	100.00	100.00
fraud	69.83	4.66	19.00	20.41	18.29	7.98
Wilt	77.46	54.80	63.19	65.72	58.26	59.37
WDBC	96.43	95.68	82.15	82.65	86.01	87.12
cardio	78.92	43.23	69.66	66.78	70.37	64.22
campaign	34.99	30.21	41.63	42.52	36.73	33.56
yeast	51.00	48.62	49.43	49.57	49.17	48.82
PageBlocks	62.67	41.93	59.35	60.27	47.40	48.77
backdoor	46.86	50.90	52.68	57.78	57.44	36.94
WPBC	66.80	84.04	87.00	86.81	84.49	85.20
smtp	41.99	7.74	39.46	37.76	25.67	36.51
ALOI	6.98	6.72	6.81	6.85	6.81	6.72
shuttle	99.25	95.76	99.18	99.03	99.04	98.69
mnist	57.97	62.20	64.14	62.84	58.56	61.32
speech	2.95	2.87	2.97	2.80	3.85	3.23
pendigits	66.33	48.16	69.88	66.86	64.69	60.61
magic.gamma	87.78	88.42	89.32	89.12	89.45	89.50
Lymphography	100.00	100.00	100.00	100.00	100.00	100.00
annthyroid	48.81	42.62	43.45	43.28	41.93	41.30
glass	89.85	92.93	97.11	95.95	95.86	95.97
satellite	88.05	85.05	84.59	84.52	86.06	85.54
http	93.37	38.34	55.41	50.43	43.16	63.44
cover	83.24	77.44	86.62	86.95	84.54	83.20
satimage-2	92.78	71.77	88.87	92.83	92.80	89.31
Pima	80.21	77.09	78.55	78.26	78.36	78.36
celeba	7.88	6.50	7.26	7.99	8.41	6.01
SpamBase	80.99	84.19	88.31	87.41	88.05	87.24
letter	9.33	21.53	10.43	10.52	11.75	12.42
donors	98.23	98.45	99.72	98.05	98.97	99.22
wine	100.00	100.00	100.00	100.00	100.00	100.00
CIFAR10_0	23.66	22.35	23.79	24.95	25.39	26.56
CIFAR10_1	12.24	13.71	16.01	16.19	14.44	15.26
CIFAR10_2	13.02	12.26	14.86	14.94	15.32	14.20
CIFAR10_3	12.85	9.59	13.21	13.56	12.45	11.67
CIFAR10_4	23.34	22.17	31.35	32.12	30.53	31.52
CIFAR10_5	10.94	8.93	12.04	11.99	11.88	9.82
CIFAR10_6	15.99	13.37	20.87	21.08	20.19	19.46
CIFAR10_7	18.49	16.37	23.92	24.91	23.57	21.06
CIFAR10_8	18.32	20.03	20.46	21.91	20.78	19.34
CIFAR10_9	21.14	21.88	24.01	23.55	22.57	24.08

Continued on next page

datasets	FoMo-0D	1st base model	1st Bagging	2nd Bagging	3rd Bagging	1st Selected Bagging
FashionMNIST_0	42.01	34.50	52.71	53.59	54.54	51.22
FashionMNIST_1	59.22	46.55	88.40	88.05	86.06	86.68
FashionMNIST_2	36.08	22.63	48.27	46.75	47.29	42.62
FashionMNIST_3	40.80	39.58	64.76	61.91	64.51	61.46
FashionMNIST_4	42.82	31.50	52.51	54.27	50.14	47.52
FashionMNIST_5	70.42	58.37	85.30	86.19	86.18	85.01
FashionMNIST_6	27.96	21.85	32.31	30.97	34.40	30.75
FashionMNIST_7	84.35	61.71	90.89	90.14	90.85	87.59
FashionMNIST_8	33.22	39.93	40.95	41.55	43.21	43.17
FashionMNIST_9	77.90	69.86	88.53	87.52	89.09	86.15
MNIST-C_rotate	10.70	13.48	13.76	14.46	13.76	13.60
MNIST-C_brightness	21.58	36.02	44.42	39.72	44.16	44.68
MNIST-C_scale	15.68	54.75	56.37	56.88	56.39	60.63
MNIST-C_spatter	52.32	58.29	73.01	73.84	70.91	73.84
MNIST-C_shear	18.15	19.88	25.52	26.98	23.17	22.80
MNIST-C_translate	16.60	21.68	26.31	28.37	25.80	26.01
MNIST-C_shot_noise	32.47	24.43	55.22	55.05	51.03	48.69
MNIST-C_zigzag	39.23	42.23	68.47	67.83	66.47	67.53
MNIST-C_identity	8.73	9.05	8.71	8.42	8.46	8.32
MNIST-C_dotted_line	31.46	30.21	59.38	58.61	56.70	57.76
MNIST-C_motion_blur	43.69	58.30	76.60	77.12	76.23	75.46
MNIST-C_glass_blur	70.65	51.98	94.89	93.77	92.72	91.47
MNIST-C_impulse_noise	99.91	94.05	99.98	100.00	99.99	99.74
MNIST-C_fog	62.76	67.99	90.79	90.20	91.63	92.56
MNIST-C_canny_edges	37.77	50.63	53.65	50.82	58.00	55.64
MNIST-C_stripe	99.51	88.19	99.75	99.79	99.79	99.23
MVTec-AD_pill	77.34	78.90	86.26	85.78	86.08	86.20
MVTec-AD_cable	70.00	87.72	86.82	86.73	89.08	86.22
MVTec-AD_capsule	66.87	82.95	89.00	89.09	90.54	88.83
MVTec-AD_leather	96.47	90.95	98.39	97.36	98.65	94.90
MVTec-AD_hazelnut	54.76	66.32	78.45	77.60	75.98	79.59
MVTec-AD_wood	75.27	86.05	92.39	92.56	92.40	90.76
MVTec-AD_carpet	66.55	78.84	82.94	83.43	82.89	82.89
MVTec-AD_metal_nut	76.82	81.27	91.38	90.52	89.80	88.48
MVTec-AD_transistor	55.85	73.90	89.30	91.42	88.42	89.32
MVTec-AD_zipper	90.16	91.49	95.58	95.82	96.13	96.00
MVTec-AD_grid	54.86	75.24	82.64	82.31	82.85	79.63
MVTec-AD_toothbrush	80.45	93.23	98.58	98.29	98.50	97.81
MVTec-AD_screw	43.16	63.60	73.61	72.88	73.83	73.93
MVTec-AD_bottle	96.70	99.32	98.92	98.69	98.47	98.48
MVTec-AD_tile	82.11	95.98	94.84	94.97	95.29	95.78
SVHN_0	11.36	10.65	16.39	17.00	17.04	17.23
SVHN_1	12.67	11.51	15.62	15.60	15.20	14.71
SVHN_2	18.98	18.13	19.71	19.06	19.51	18.56
SVHN_3	15.56	16.91	16.13	15.90	16.63	16.25
SVHN_4	16.47	17.98	16.33	15.93	17.67	16.73
SVHN_5	17.90	18.30	20.11	18.98	19.12	18.46
SVHN_6	13.35	12.75	13.67	14.05	13.85	14.13
SVHN_7	13.96	15.60	17.55	17.43	18.13	16.64
SVHN_8	13.56	12.83	16.25	15.57	15.96	14.98
SVHN_9	13.68	12.00	13.17	13.91	13.88	14.32
agnews_0	12.20	14.97	17.51	16.60	17.81	18.74
agnews_1	15.58	22.66	28.29	29.77	28.65	26.90
agnews_2	17.46	28.22	27.74	28.28	28.57	27.39
agnews_3	12.05	18.69	21.56	21.11	22.28	18.72
20news_0	15.58	22.25	26.57	26.39	26.09	23.64
20news_1	9.79	10.72	12.45	12.97	12.47	11.51
20news_2	10.02	9.85	10.73	10.72	10.63	10.16
20news_3	28.26	23.98	31.09	29.73	28.76	29.92
20news_4	12.31	12.53	12.68	12.32	14.36	13.96
20news_5	11.28	11.33	14.97	14.15	18.46	14.03
amazon	9.48	10.69	11.14	11.02	11.50	12.40
imdb	8.28	9.04	9.02	8.90	9.51	9.57
yelp	12.38	15.18	15.66	14.97	15.17	16.84

A.2.3 Average F1 scores

Table A.4: Average F1 scores of the FoMo-0D, our top-1 base model, top-3 Bagging ensembles and top-1 Selected Bagging ensemble across all ADBench datasets.

datasets	FoMo-0D	1st base model	1st Bagging	2nd Bagging	3rd Bagging	1st Selected Bagging
mammography	37.69	39.62	45.00	44.62	42.31	41.54
vowels	28.00	6.00	28.00	22.00	26.00	28.00
optdigits	37.33	26.00	45.33	43.33	36.00	36.67
thyroid	59.14	62.37	61.29	63.44	63.44	63.44
WBC	95.65	95.65	89.13	89.13	84.78	84.78
Stamps	88.61	87.34	89.87	88.61	89.87	84.81
Cardiotocography	56.22	49.14	48.71	48.93	49.57	49.36
fault	57.50	56.61	55.72	55.57	56.02	55.42
census	13.48	14.90	12.67	12.71	12.58	14.12
skin	51.09	82.30	86.80	86.64	85.17	85.90
InternetAds	49.18	68.48	63.32	59.51	63.04	59.78
breastw	97.29	89.46	97.29	96.99	96.69	96.08
Waveform	8.00	9.00	12.00	13.00	11.00	11.00
vertebral	63.37	81.19	79.21	79.21	77.23	77.23
landsat	52.66	49.74	50.41	50.34	50.86	51.39
musk	96.91	0.00	42.27	71.13	60.82	41.24
Ionosphere	92.33	91.15	93.22	92.92	92.33	92.33
Hepatitis	100.00	100.00	100.00	100.00	100.00	100.00
fraud	76.09	6.52	25.00	17.39	17.39	5.43
Wilt	77.04	61.09	66.93	66.54	62.26	61.87
WDBC	91.30	82.61	91.30	91.30	82.61	82.61
cardio	72.16	45.45	69.89	69.32	68.75	71.02
campaign	40.00	33.45	42.78	44.12	39.14	36.83
yeast	50.49	47.34	48.13	48.13	46.55	46.94
PageBlocks	61.18	49.02	57.45	57.84	54.31	52.55
backdoor	43.08	53.91	65.72	69.22	64.58	42.67
WPBC	57.63	85.17	89.41	89.41	88.56	87.71
smtp	33.33	14.29	33.33	33.33	33.33	33.33
ALOI	7.76	8.29	8.29	7.96	8.82	8.75
shuttle	97.86	96.58	98.15	98.18	98.12	97.86
mnist	62.00	63.86	62.86	61.29	62.29	61.14
speech	3.28	0.00	3.28	3.28	6.56	1.64
pendigits	69.23	49.36	78.21	76.28	73.08	73.08
magic.gamma	78.12	79.68	80.28	80.05	80.58	80.43
Lymphography	100.00	100.00	100.00	100.00	100.00	100.00
annthyroid	49.25	45.13	44.38	44.76	44.19	44.38
glass	76.00	90.00	96.00	96.00	94.00	96.00
satellite	77.75	73.92	73.18	73.18	74.41	74.26
http	92.86	4.08	16.33	4.08	4.08	46.43
cover	81.25	77.59	81.25	81.47	78.45	77.16
satimage-2	88.73	69.01	84.51	87.32	85.92	80.28
Pima	74.46	72.31	74.19	74.46	73.66	73.92
celeba	7.16	9.43	9.80	10.37	10.84	7.16
SpamBase	74.27	79.21	79.69	79.09	80.64	80.58
letter	4.00	23.00	5.00	6.00	7.00	12.00
donors	98.73	98.28	99.31	99.14	99.59	99.59
wine	100.00	100.00	100.00	100.00	100.00	100.00
CIFAR10_0	27.38	27.00	27.38	28.14	28.90	28.14
CIFAR10_1	10.65	15.21	19.01	20.15	15.21	16.73
CIFAR10_2	12.55	14.07	20.53	18.63	19.01	17.11
CIFAR10_3	13.69	10.27	15.97	15.97	15.21	12.55
CIFAR10_4	27.76	28.14	36.12	36.88	34.98	35.74
CIFAR10_5	10.27	5.32	14.07	12.93	12.17	9.89
CIFAR10_6	17.49	14.45	22.81	24.71	22.43	22.43
CIFAR10_7	22.81	20.53	26.24	25.48	25.86	22.81
CIFAR10_8	20.91	25.48	27.38	26.62	23.95	22.43
CIFAR10_9	26.24	27.38	27.76	25.48	27.00	29.66

Continued on next page

datasets	FoMo-0D	1st base model	1st Bagging	2nd Bagging	3rd Bagging	1st Selected Bagging
FashionMNIST_0	45.71	40.32	58.10	58.10	59.05	55.56
FashionMNIST_1	58.41	52.70	83.17	83.49	82.22	80.95
FashionMNIST_2	41.27	28.25	52.70	52.38	51.11	48.57
FashionMNIST_3	45.71	46.98	65.71	63.17	64.13	61.59
FashionMNIST_4	47.94	36.51	50.79	52.70	51.75	49.21
FashionMNIST_5	71.75	54.92	79.37	80.00	78.41	79.68
FashionMNIST_6	35.24	27.30	43.81	42.54	44.13	39.37
FashionMNIST_7	79.68	58.41	83.81	83.81	84.13	83.81
FashionMNIST_8	36.83	44.13	46.35	44.76	43.81	46.03
FashionMNIST_9	68.57	67.30	80.32	79.37	81.59	79.37
MNIST-C_rotate	12.60	15.80	16.00	17.00	16.80	16.80
MNIST-C_brightness	24.20	37.20	44.20	42.60	45.60	45.60
MNIST-C_scale	15.00	54.80	57.60	60.00	56.80	59.80
MNIST-C_spatter	50.00	55.00	67.00	68.20	65.40	67.80
MNIST-C_shear	24.00	24.60	30.20	33.00	30.20	27.00
MNIST-C_translate	21.40	27.00	31.20	33.00	29.80	32.00
MNIST-C_shot_noise	35.80	26.60	53.80	55.60	51.80	49.20
MNIST-C_zigzag	44.00	39.80	63.40	62.40	61.60	62.20
MNIST-C_identity	7.80	9.00	8.00	7.80	7.60	6.60
MNIST-C_dotted_line	34.20	31.80	57.00	57.40	55.80	57.80
MNIST-C_motion_blur	42.60	55.00	70.80	70.00	69.80	68.40
MNIST-C_glass_blur	64.80	50.00	87.80	86.60	85.20	83.40
MNIST-C_impulse_noise	99.00	88.20	99.40	99.80	99.60	99.40
MNIST-C_fog	58.00	65.20	84.00	83.80	84.20	86.40
MNIST-C_canny_edges	39.80	46.20	52.60	51.00	55.80	56.00
MNIST-C_stripe	97.40	79.00	98.80	99.00	98.20	98.20
MVTec-AD_pill	69.59	80.41	80.70	80.99	80.70	81.87
MVTec-AD_cable	59.17	81.19	79.82	78.90	80.28	81.19
MVTec-AD_capsule	52.61	73.20	83.01	83.33	84.97	83.99
MVTec-AD_leather	93.63	92.51	96.63	96.25	96.25	95.51
MVTec-AD_hazelnut	46.43	67.14	68.57	67.86	64.29	68.57
MVTec-AD_wood	64.32	80.90	83.42	83.42	82.91	81.41
MVTec-AD_carpet	53.12	74.55	75.89	78.12	74.11	75.45
MVTec-AD_metal_nut	69.60	75.46	83.15	82.42	83.52	81.32
MVTec-AD_transistor	50.00	72.88	80.51	85.59	82.20	80.51
MVTec-AD_zipper	81.87	90.35	94.15	94.44	94.15	93.57
MVTec-AD_grid	47.88	73.94	80.00	78.18	75.15	73.94
MVTec-AD_toothbrush	67.38	82.27	92.91	92.91	91.84	89.72
MVTec-AD_screw	36.96	63.04	71.74	70.00	70.87	72.61
MVTec-AD_bottle	91.04	95.02	93.53	93.53	93.53	95.52
MVTec-AD_tile	74.15	86.86	85.59	85.59	84.75	86.02
SVHN_0	10.00	11.92	19.23	21.15	20.77	20.77
SVHN_1	14.20	12.80	18.60	15.80	17.40	16.20
SVHN_2	22.60	21.80	23.60	22.40	22.60	23.20
SVHN_3	19.02	22.60	20.36	19.24	19.24	20.58
SVHN_4	20.66	23.72	19.90	19.39	21.94	21.43
SVHN_5	21.55	22.93	23.48	22.65	22.38	22.10
SVHN_6	18.60	14.95	19.27	18.94	19.93	18.94
SVHN_7	17.01	20.07	24.49	24.49	23.47	19.73
SVHN_8	18.49	16.23	18.87	19.62	17.36	18.11
SVHN_9	17.14	15.92	19.18	18.37	18.37	17.55
agnews_0	14.00	17.20	19.60	20.20	20.80	21.60
agnews_1	18.80	27.00	33.80	35.80	33.40	32.60
agnews_2	22.20	30.80	30.60	30.60	33.40	32.20
agnews_3	14.40	25.40	26.20	25.40	27.20	22.40
20news_0	19.48	27.27	33.12	33.12	33.12	31.17
20news_1	9.60	12.80	14.40	14.40	12.80	13.60
20news_2	10.48	8.06	9.68	11.29	8.06	9.68
20news_3	28.89	26.67	28.89	26.67	24.44	20.00
20news_4	13.41	10.98	13.41	13.41	18.29	15.85
20news_5	9.21	17.11	15.79	17.11	17.11	15.79
amazon	8.40	11.20	11.60	10.40	13.00	13.20
imdb	6.00	8.80	7.00	7.40	8.00	6.80
yelp	14.20	18.00	18.00	18.80	18.40	20.40

Eidesstattliche Versicherung

(Affidavit)

Nguyen, Phuong Huong

Name, Vorname
(surname, first name)

216601

Matrikelnummer
(student ID number)

☐ Bachelorarbeit
(Bachelor's thesis)

■ Masterarbeit
(Master's thesis)

Titel
(Title)

Investigating Ensemble Strategies for Enhancing the Generalization of Foundation Models in Outlier Detection

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem oben genannten Titel selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

I declare in lieu of oath that I have completed the present thesis with the above-mentioned title independently and without any unauthorized assistance. I have not used any other sources or aids than the ones listed and have documented quotations and paraphrases as such. The thesis in its current or similar version has not been submitted to an auditing institution before.

Dortmund, 10.05.2026

Ort, Datum
(place, date)

Unterschrift
(signature)

Belehrung:
Wer vorsätzlich gegen eine die Täuschung über Prüfungsleistungen betreffende Regelung einer Hochschulprüfungsordnung verstößt, handelt ordnungswidrig. Die Ordnungswidrigkeit kann mit einer Geldbuße von bis zu 50.000,00 € geahndet werden. Zuständige Verwaltungsbehörde für die Verfolgung und Ahndung von Ordnungswidrigkeiten ist der Kanzler/die Kanzlerin der Technischen Universität Dortmund. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuches kann der Prüfling zudem exmatrikuliert werden. (§ 63 Abs. 5 Hochschulgesetz - HG -).

Die Abgabe einer falschen Versicherung an Eides statt wird mit Freiheitsstrafe bis zu 3 Jahren oder mit Geldstrafe bestraft.

Die Technische Universität Dortmund wird ggf. elektronische Vergleichswerkzeuge (wie z.B. die Software „turnitin") zur Überprüfung von Ordnungswidrigkeiten in Prüfungsverfahren nutzen.

Die oben stehende Belehrung habe ich zur Kenntnis genommen:

Official notification:
Any person who intentionally breaches any regulation of university examination regulations relating to deception in examination performance is acting improperly. This offense can be punished with a fine of up to EUR 50,000.00. The competent administrative authority for the pursuit and prosecution of offenses of this type is the Chancellor of TU Dortmund University. In the case of multiple or other serious attempts at deception, the examinee can also be unenrolled, Section 63 (5) North Rhine-Westphalia Higher Education Act (*Hochschulgesetz, HG*).

The submission of a false affidavit will be punished with a prison sentence of up to three years or a fine.

As may be necessary, TU Dortmund University will make use of electronic plagiarism-prevention tools (e.g. the "turnitin" service) in order to monitor violations during the examination procedures.

I have taken note of the above official notification:*

Dortmund, 10.05.2026

Ort, Datum
(place, date)

Unterschrift
(signature)

***Please be aware that solely the German version of the affidavit ("Eidesstattliche Versicherung") for the Bachelor's/ Master's thesis is the official and legally binding version.**